

Web Dev to SAP + AI Engineer

A structured self-study roadmap: Foundation to job-ready by Jan/Feb 2027

PART I [YOU ARE HERE] 50-Day Foundation — Jul 13 to Sep 14, 2026 (incl. JS reset week)

PART II SAP Overview (Sep 15-17) + ABAP Institute (Sep 18-20 start → end Nov / mid Dec)

PART III SAP BTP + AI — self-paced, begins Dec 1, 2026

PART IV Job Hunt — Jan/Feb 2027 onwards

49

STUDY DAYS

12

PHASES

8

BREAK DAYS

Sep 14

COMPLETION

Nikhil Vanama — Career Roadmap 2026

Web Developer → SAP + AI Engineer • Jul 2026 — Jan/Feb 2027 • *Click any row to jump to its page*

SECTION	TOPIC	DATES
PART I	50-Day Foundation — Master Timeline	Jul 13 - Sep 14
	Daily Timings — weekday + weekend rhythm	—
	Mindset & Method — how to learn effectively	—
	Prep Week — Resource Collection	Jul 8-12, 2026
	Phase 1: Prompt Engineering	Jul 13-15
	Phase 2: Claude Code CLI	Jul 16-17
	Phase 3: CS Fundamentals	Jul 18-19
RESET	JS Reset / Revision Week — revise P1-3, warm up JS	Jul 20-26, 2026
	Phase 4: JavaScript Basics Warm-up	Jul 27-28
	Phase 5: Advanced JavaScript + OOP	Jul 29 - Aug 1
	Phase 6: React + Tailwind CSS	Aug 3-8
	Phase 7: Node.js + Express (Deep Dive)	Aug 10-18
	Phase 8: REST API Design	Aug 19-23
	Phase 9: MySQL + Node Integration	Aug 24 - Sep 1
	Phase 10: Git (Advanced) + Docker + AWS	Sep 2-5
	Full-Stack Capstone A — Deploy the Auth API	weekends + breaks
	Phase 11: System Design Fundamentals	Sep 6-9
	Full-Stack Capstone B — Scale + Portfolio Polish	final weekends
	Phase 12: Agentic AI + n8n + FocusTrack Pro	Sep 10-14
	Agentic Capstone C — Personal Research Agent	final weekends
	Agentic Capstone D — RAG Assistant + Automations	final weekends
PART II	Part II Overview — SAP Ecosystem + ABAP Institute	Sep 15 → end Nov / mid Dec
	Phase A: SAP Overview & Ecosystem (self-study)	Sep 15-17
	Institute Orientation — kickoff	Sep 18-20
	Segment 1: ABAP Workbench & Language Basics (M1-M4)	~Week 1
	Segment 2: ABAP Dictionary (DDIC) (M5-M8)	~Week 2
	Segment 3: Search Helps, Locks, Open SQL (M9-M12)	~Week 3

SECTION	TOPIC	DATES
	Segment 4: Internal Tables, Debugging, Sel Screens (M13-M16)	~Week 4
	Segment 5: Modularization (M17-M20)	~Week 5
	Segment 6: Reports — Classical, Interactive, ALV (M21-M24)	~Week 6
	Segment 7: Dialog Programming / Module Pool (M25-M28)	~Week 7
	Segment 8: OO ABAP (M29-M31)	~Week 8
	Segment 9: BDC & LSMW (M32-M34)	~Week 9
	Segment 10: Scripts, Smart Forms, RFC, ALE, IDocs (M35-M37)	~Week 10
	Segment 11: BAPIs, Exits, BADIs, CTS + SAPUI5 (M38-M39)	~Week 11
	Wrap: ABAP on HANA — CDS, AMDP (M40)	Final week
PART III	SAP BTP + AI Self-Pace — overview	Dec 1, 2026 →
	Phase 13: BTP Foundation	Dec 1-7
	Phase 14: CAP Framework (backend)	Dec 8-15
	Phase 15: SAP AI Core + Joule + RAG	Dec 16-23
	Phase 16: SBPA + Smart Vendor Hub Capstone	Dec 24-31
PART IV	Job Hunt Strategy	Jan/Feb 2027
	Resume, Portfolio & LinkedIn Strategy	Week 1
	90-Question Interview Bank	Week 3

Master Timeline — 49-Day Foundation

Jul 13, 2026 → Sep 14, 2026 • 49 study days + 8 break days + 1 JS reset week = 64 calendar days • Most breaks fall on weekends

#	PHASE	DAYS	DATES	FOCUS
-	Prep Week	5	Jul 8-12	Resource collection; Tirumala Jul 11-12
1	Prompt Engineering	3	Jul 13-15 (Mon-Wed)	5-element anatomy, zero/few-shot, CoT, refinement
2	Claude Code CLI	2	Jul 16-17 (Thu-Fri)	CLAUDE.md, slash commands, MCP, plan mode, agent workflows
3	CS Fundamentals	2	Jul 18-19 (Sat-Sun)	CPU/RAM, binary, OSI, HTTP, DNS, TCP, data structures, Big O — full weekend
↺	JS Reset / Revision Week	7	Jul 20-26 (Mon-Sun)	Catch-up buffer — revise Phases 1-3, redo DS/closures from memory, warm up JS. Evenings: start institute shortlisting.
4	JS Basics Warm-up	2	Jul 27-28 (Mon-Tue)	Variables, types, functions, arrays, objects, DOM
5	Advanced JavaScript + OOP	4	Jul 29 - Aug 1 (Wed-Sat)	Closures, prototypes, classes, inheritance, polymorphism, async/await, event loop
BREAK DAY — Aug 2 (Sun) — Revise Phase 5 Advanced JS + OOP				
6	React + Tailwind CSS	6	Aug 3 - Aug 8 (Mon-Sat)	JSX, hooks, Router v6, context, useReducer, Tailwind
BREAK DAY — Aug 9 (Sun) — Revise Phase 6 React				
7	Node.js + Express (Deep)	9	Aug 10-18 (Mon-Tue)	Runtime, middleware, auth, JWT, validation, MVC, file uploads
8	REST API Design	3	Aug 19-21 (Wed-Fri)	URL design, status codes, pagination, versioning, OpenAPI
BREAK DAYS — Aug 22-23 (Sat-Sun) — Revise Phases 7-8 — Full Weekend Off				
9	MySQL + Node Integration	7	Aug 24-30 (Mon-Sun)	DDL, DML, JOINS, indexes, normalization, mysql2 driver
BREAK DAYS — Aug 31 - Sep 1 (Mon-Tue) — Revise Phase 9 MySQL				
10	Git (Advanced) + Docker + AWS	3	Sep 2-4 (Wed-Fri)	Rebase, stash, Docker, docker-compose, EC2, S3, Lambda
BREAK DAY — Sep 5 (Sat) — Rest after Git/Docker/AWS — push all Phase 10 code, review deploy checklist				

#	PHASE	DAYS	DATES	FOCUS
11	System Design Fundamentals	3	Sep 6 - Sep 8 (Sun-Tue)	Scaling, caching, CDN, queues, CAP theorem, sharding
BREAK DAY — Sep 9 (Wed) — Revise Phases 10-11				
12	Agentic AI + n8n + Capstone	5	Sep 10-14 (Thu-Mon)	Claude API, tool use, RAG, n8n workflows, FocusTrack Pro

Total: 49 study days + 8 break days + 1 JS reset week (Jul 20-26) = 64 calendar days. The reset week absorbs the missed JS block; JS restarts fresh on Jul 27. CS Fundamentals lands on Sat-Sun so all subsequent break days naturally land on weekends.

WEEKLY TIME BUDGET

- Weekdays (Mon-Fri): 2.5 hrs x 5 = **12.5 hrs**
- Weekends (Sat+Sun): 8 hrs x 2 = **16 hrs** (5 morning + 3 project)
- **Total per week: ~28 hrs**
- Over 8 weeks: **220+ focused hours**

BREAK DAY ACTIVITY (ALL BREAKS)

- Re-read notes from completed phase
- Redo 2-3 practice questions from memory
- Update GitHub with completed code
- Log the break in your daily organiser app

Tracker tip: You already built a daily organiser app for tasks / goals / routines. Add one checklist per phase to it and tick off each day — visible streak = psychological locking. Don't break the chain.

Your Actual Daily Timings

Weekday office 12–9 PM • Weekends longer study block • Fixed windows so you never negotiate with yourself

WEEKDAYS (MON – FRI) • OFFICE 12–9 PM

7:15 AM	Wake up
7:20 – 8:15 AM	Workout (55 min)
8:20 – 8:45 AM	Freshup, breakfast
8:45/9:00 – 11:20 AM	DEEP STUDY BLOCK — ~2.5 hrs (highest quality window)
11:20 AM	Start getting ready for office
12:00 PM – 9:00 PM	Office (Mediacle)
9:00 – 10:00 PM	Commute + dinner
10:00 – 11:15/11:30 PM	Personal work — freelance, revision of morning notes, SATA maintenance

The 8:45–11:20 AM slot is where the roadmap topics happen. The 10–11:30 PM slot is *only* for revision / freelance / your daily organiser app — not for new topics.

WEEKENDS (SATURDAY + SUNDAY)

9:00/9:30 AM – 2:00 PM	MAIN STUDY BLOCK — 4.5-5 hrs (longest single session)
2:00 – 3:00 PM	Lunch break
3:00 – 6:00 PM	If outing planned: no study today. If not: movie / BGMI break (guilt-free)
6:00 – 9:00 PM	PROJECT BLOCK — 3 hrs (use mainly for capstone / mini-project completion)

Weekends are your project-completion runway. Weekday morning slots deliver theory + small exercises; weekends stitch it into a working project.

NON-NEGOTIABLES

- Morning 8:45 study — even if only 45 min
- Phone on DND, one browser tab only
- No new topic after 10 PM — only revision
- Sunday evening: plan the week ahead

IF OFFICE RUNS LATE

- Skip 10–11 PM personal work, sleep by 11
- Never skip morning to compensate
- Log the disruption in your organiser app
- Weekend absorbs the debt, not the next weekday

How to Stay Consistent & Learn Effectively

Read this when you feel like skipping. Refer back whenever motivation dips.

RULE #1 — THE 1% RULE

You don't need 8 hours a day. **2.5 focused hours beats 8 distracted hours.** Every day you study, you are 1% better than yesterday. In 8 weeks, that compounds. Don't chase marathon sessions — chase daily consistency.

FEYNMAN TECHNIQUE

After each topic, **explain it in simple words as if teaching a 10-year-old.** If you can't explain it simply, you don't know it yet — re-read.

"If you can't explain it simply, you don't understand it well enough." — Feynman

MORNING = GOLD TIME

Your **8:45 – 11:20 AM slot** is your highest quality window. Phone on DND, one browser tab only, no rabbit holes. Even if sleepy — sit down at 8:45; it passes in 10 min.

WHEN YOU FEEL LIKE SKIPPING

- Every expert was once a beginner — you're building the foundation
- Slot is already blocked — just open the laptop
- Don't break the chain — tick your daily organiser app
- **On bad days: revision beats new content**

CONSISTENCY FORMULA

Weekdays: 2.5 hrs x 5 = **12.5 hrs**

Weekends: (5 + 3) x 2 = **16 hrs**

~28 hrs/week • 8 weeks = 220+ focused hours

6-STEP METHOD TO UNDERSTAND ANY CONCEPT

STEP 1 — PREVIEW

Watch a 5-10 min overview video
FIRST for the big picture

STEP 2 — STUDY

Read the concept in depth from the
recommended resource

STEP 3 — CODE

Type a small example from scratch
— no copy-paste

STEP 4 — BREAK

Change a value, remove a line,
observe what breaks and why

STEP 5 — TEACH

Explain aloud (Feynman check).
Write in your notes plainly

STEP 6 — CONNECT

Link to what you already know (e.g.,
WP hooks → React useEffect)

Your web dev background is your superpower — you already know more than you think

Prep Week — Resource Collection Mode

July 8-12, 2026 • Note: July 11-12 = Tirumala Darshan

This is NOT active coding or studying. Spend these 5 days collecting and organizing resources for every phase so you can hit the ground running on July 13.

What to do (Jul 8-10)

- Open each phase below and note the YouTube playlists listed
- Download or bookmark all videos/docs for offline use
- Create one folder per phase in your notes app
- Set up GitHub repo: `nikhil-roadmap-2026`
- *Already installed:* VS Code + Claude Code extension (Max plan), Node.js, Git, Docker — skip these
- Only new tool needed: MySQL Workbench (for Phase 9, install now to save time)
- `n8n` comes later — install with `npx n8n` at Phase 12

Jul 11-12 (Tirumala)

- Travel and darshan — complete rest from study
- Optional: read the Phase 1 Prompt Engineering overview on phone (5 min)
- Come back refreshed and ready for Jul 13 Monday start

Phase 1 starts Monday Jul 13. You are ready.

Resource Checklist

- | | |
|---|--|
| ☐ Bookmarked Anthropic Prompt Engineering docs | ☐ Saved "Docker Tutorial" (TechWorld with Nana) |
| ☐ Bookmarked Claude Code official docs | ☐ Saved "System Design" (ByteByteGo channel) |
| ☐ Saved "Namaste JavaScript" playlist (Akshay Saini) | ☐ GitHub repo created and cloned locally |
| ☐ Saved "React 18 Full Course" (The Net Ninja) | ☐ VS Code + extensions installed (ESLint, Prettier, REST Client) |
| ☐ Saved "Node.js Crash Course" (Traversy Media) | ☐ Node.js LTS + MySQL Workbench installed |
| ☐ Saved "MySQL Tutorial" (Programming with Mosh) | ☐ Claude API key obtained from <code>console.anthropic.com</code> |

Prompt Engineering

3 days • Jul 13 (Mon) – Jul 15 (Wed) • No prior AI knowledge needed

What is this? The craft of writing precise instructions that get exactly the output you want from Claude / GPT / any LLM. Prompts are the "code" of the AI era.

Why should you learn this first? Every phase after this uses AI heavily (Claude Code, RAG, capstone). A great prompt turns a 4-hour task into 20 minutes. A bad prompt wastes tokens and time. This is the meta-skill that multiplies everything else.

Topics covered: 5-element prompt anatomy (Role, Context, Task, Format, Constraints); zero-shot; few-shot with examples; chain-of-thought reasoning; role prompting; structured JSON output; iterative refinement

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Mon Jul 13 5-Element Anatomy + Zero-Shot

CONCEPTS

- The 5 elements of a strong prompt: **Role, Context, Task, Format, Constraints**
- Zero-shot = ask directly, no examples — works when the task is common (translate, summarise, classify)
- When zero-shot fails: ambiguous instructions, novel tasks, formatting-sensitive output
- How to score a prompt: clarity + specificity + verifiability of output
- Anti-patterns: vague roles ("be helpful"), missing constraints, no output format specified

READ / WATCH

Anthropic Prompt Engineering docs (Intro + Basics sections); "ChatGPT Prompt Engineering" DeepLearning.AI Ch.1-2.

HOW TO PRACTICE

Write 5 zero-shot prompts for different tasks. Evaluate and score them 1–5 for clarity.

MINI CODE SNIPPET

```
# Example prompt (5 elements)
Role:    You are a senior technical writer.
Context: Users read release notes on mobile in <30s.
Task:    Rewrite the changelog below in 3 bullets.
Format:  Markdown bullets, no headings.
Constr:  Under 40 words total. No jargon.
<changelog>{{input}}</changelog>
```

DAY 2 Tue Jul 14 Few-Shot + Chain-of-Thought

CONCEPTS

- Few-shot = 2–5 input/output examples shown before the real question
- Examples must be diverse (edge cases) + short (token cost) + consistent format
- Chain-of-Thought (CoT) = ask the model to reason step by step before answering
- CoT shines on math, logic, multi-step debugging — not on lookups

- Zero-shot CoT trick: just append "Let's think step by step" to a plain prompt

READ / WATCH

Anthropic docs (Examples section); DeepLearning.AI Ch.3–4; read OpenAI "Chain of Thought" paper (abstract + examples only).

HOW TO PRACTICE

Take 3 complex tasks (code debug, reasoning, summarization) and write both few-shot and CoT prompts. Compare outputs.

MINI CODE SNIPPET

```
# Few-shot classifier prompt
Classify sentiment. Examples:
"Loved the update" → POSITIVE
"Battery drained fast" → NEGATIVE
"Works ok I guess" → NEUTRAL

Now classify: "{{text}}"

# CoT variant: add "Let's think step by step" before answering
```

DAY 3 Wed Jul 15 Structured Outputs + Iterative Refinement + Role Prompting

CONCEPTS

- JSON output: describe the schema in the prompt + include "Return ONLY valid JSON"
- For strict schemas use tool use / function calling instead of raw JSON
- Role prompting: You are a senior SQL DBA. Reply ONLY with valid MySQL 8 queries
- Iterative refinement loop: run → grade output → add missing constraint → rerun
- Version your prompts (v1, v2, v3) — keep the diffs in a "prompt library" file

READ / WATCH

Anthropic docs (Output Formatting, Role Prompting sections); review your Day 1–2 prompts.

HOW TO PRACTICE

Build a "prompt library" document with 10 reusable prompt templates. Practice refining 1 prompt through 5 iterations.

MINI CODE SNIPPET

```
# Force JSON output
Return ONLY valid JSON matching this schema:
{ "title": string,
  "priority": "low"|"med"|"high",
  "tags": string[] }

Input: "{{task}}"
```

YOUTUBE + DOCS (OPEN THESE DIRECTLY)

- YT** ChatGPT Prompt Engineering for Developers — DeepLearning.AI (free): <https://learn.deeplearning.ai/chatgpt-prompt-eng>
- YT** Prompt Engineering Tutorial (freeCodeCamp, 41 min): https://youtu.be/_ZvnD73m40o
- YT** Anthropic Prompt Engineering Course (playlist): <https://www.youtube.com/@AnthropicAI>
- DOCS** Anthropic docs (end to end): <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

Recommended certification: "ChatGPT Prompt Engineering for Developers" — DeepLearning.AI (free, ~1 hr, includes certificate). Also: Vanderbilt "Prompt Engineering for ChatGPT" on Coursera (free audit).

PRACTICE QUESTIONS

Q1 —

Write a zero-shot prompt that converts raw meeting notes into a structured action-items list with owner, deadline, and priority.

Q2 —

Create a few-shot prompt (3 examples) that transforms casual Slack messages into formal professional emails.

Q3 —

Design a chain-of-thought prompt that asks Claude to debug a JavaScript function step by step, explaining each line of reasoning.

Q4 —

Write a role-based system prompt for a "SQL query assistant" that only outputs valid MySQL queries with no explanation.

Q5 —

Build a prompt with all 5 elements: Role, Context, Task, Format (JSON), Constraints (under 100 words output).

MINI PROJECT

Build a personal "Prompt Playbook" — a Markdown file with 10 high-quality reusable prompts covering: code review, bug debug, explain-like-I'm-5, API docs generator, commit message writer, test case generator, error analysis, SQL query assistant, data transformer, and study quiz generator.

DELIVERABLES CHECKLIST

- ☐ Completed 5 zero-shot prompts (Day 1)
- ☐ Compared few-shot vs CoT on 3 tasks (Day 2)
- ☐ Built 10-prompt Prompt Playbook (Day 3)
- ☐ Saved Playbook to GitHub repo
- ☐ Answered all 5 practice questions

MINI PROJECT IDEAS — PHASE 1

- Prompt Playbook** — A shareable Markdown file with 10 reusable prompts (code review, bug debug, ELI5 explainer, commit message writer, SQL query gen, test-case gen, changelog, error explainer, meeting-notes-→actions, study quiz).
- Prompt Comparator** — Node.js CLI that takes one task, runs zero-shot vs few-shot vs CoT prompts on the same input, and prints outputs side-by-side. Save results as evidence.
- Domain Assistant** — Design a single "expert" system prompt for a domain you know (SATA/community/finance) that only answers within scope and refuses politely otherwise.

Claude Code CLI

2 days • Jul 16 (Thu) – Jul 17 (Fri)

Setup status: You already run Claude Code via the VS Code extension on a Max plan. No install / auth work needed — these 2 days go straight to the *features* that will 5× the rest of the roadmap.

Why this matters: From Phase 3 onwards you'll use Claude Code every single day. Configuring memory + slash commands + MCP well now saves ~30% of coding time over the next 6 months. It also becomes a talking point in interviews ("I use AI-native tooling").

Topics covered: CLAUDE.md project memory, custom /slash commands, plan mode, MCP servers, hooks, TodoWrite, multi-step agent workflows

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Thu Jul 16 CLI Setup, CLAUDE.md, Slash Commands, Plan Mode

CONCEPTS

- You already have Claude Code in VS Code (extension + Max plan) — skip install
- CLAUDE.md = per-project memory Claude reads on every session (style, stack, guardrails)
- Custom slash commands live in `.claude/commands/*.md` (project) or `~/.claude/commands/` (global)
- Plan mode = Claude drafts a plan first, waits for approval, then executes step-by-step
- Best used for large refactors, migrations, feature scaffolding where blast radius is high

READ / WATCH

docs.anthropic.com/claude-code — Getting Started, CLAUDE.md reference, /slash commands docs; run `claude --help`.

HOW TO PRACTICE

Install Claude Code. Create a CLAUDE.md for your roadmap repo. Create 3 custom slash commands: `/explain`, `/refactor`, `/test`. Use plan mode on a sample task.

MINI CODE SNIPPET

```
# CLAUDE.md – project memory
- Use async/await; never var
- Validate every route with zod
- MySQL driver: mysql2 (never sequelize)
- Commit style: conventional (feat/fix/chore)
- Tests live in /tests mirroring src
```

DAY 2 Fri Jul 17 MCP Servers, Agent Workflows, Hooks, TodoWrite

CONCEPTS

- MCP = Model Context Protocol — standard way to give Claude tools (filesystem, GitHub, DBs)
- Configure MCP servers in `.mcp.json` or via the VS Code extension settings
- Hooks = shell commands triggered by events (pre-tool-use, post-tool-use, session-start)
- TodoWrite tool = built-in task tracker inside a Claude Code session

- Agent workflow = multi-step task where Claude picks tools in a loop until goal met

READ / WATCH

Claude Code MCP docs; Agent SDK overview; Hooks documentation.

HOW TO PRACTICE

Set up 1 MCP server (filesystem). Write a multi-step agent workflow that reads a file, analyzes it, and outputs a report. Explore the TodoWrite tool.

MINI CODE SNIPPET

```
# .claude/commands/commit.md - custom /commit
Look at staged diff. Write a conventional commit
message in imperative mood. Under 72 chars for
subject line; blank line; wrapped body at 80.
Do not include emoji or the phrase "This commit".
```

YOUTUBE + DOCS

YT Anthropic official YouTube (Claude Code walkthroughs): <https://www.youtube.com/@AnthropicAI>

DOCS Claude Code full docs: <https://docs.anthropic.com/en/docs/claude-code/overview>

DOCS MCP servers directory: <https://github.com/modelcontextprotocol/servers>

No formal certification, but do this: commit your CLAUDE.md + custom slash commands to a public repo `nikhil-claude-config`. This is your portfolio artifact for AI-tooling literacy.

PRACTICE QUESTIONS

Q1 —

Write a CLAUDE.md file for a Node.js Express project that instructs Claude to: use async/await, never use var, always add zod validation, use mysql2 for DB.

Q2 —

Create a custom /commit slash command that generates a conventional commit message from staged changes.

Q3 —

Design a 3-step agent workflow: (1) read package.json, (2) find unused dependencies, (3) output removal commands.

DELIVERABLES CHECKLIST

- ☐ Claude Code installed and authenticated
- ☐ CLAUDE.md created for nikhil-roadmap-2026 repo
- ☐ 3 custom slash commands written and tested
- ☐ MCP filesystem server configured
- ☐ Agent workflow built and working

MINI PROJECT IDEAS — PHASE 2

1. **CLAUDE.md Starter Pack** — Public repo `nikhil-claude-config` with reusable CLAUDE.md templates for Node, React, Python, ABAP-style code style guides.
2. **/commit + /review Commands** — Two custom slash commands: `/commit` generates conventional-commit messages from staged diff; `/review` reads the current file and lists 3 improvements.
3. **MCP Filesystem Report Agent** — Agent workflow: read a folder → count files by extension → find TODOs → write a Markdown report at `./report.md`.

CS Fundamentals

2 days • Jul 18 (Sat) - Jul 19 (Sun) • Weekend = deep, uninterrupted focus

What is this? Before writing code, understand what hardware, operating systems, and networks do. This is the bedrock beneath everything — JavaScript, Node.js, APIs, databases. Without this, you're building on sand.

Why should you learn this? SAP HANA runs on servers. ABAP programs run in memory. HTTP is how Fiori apps talk to SAP. Every technical interview asks about DNS, HTTP, TCP. This 2-day investment explains the WHY behind everything you code.

Topics covered: CPU/RAM/Storage, Binary, Operating Systems, Processes & Threads, Networking (IP, DNS, HTTP, HTTPS, TCP/UDP, TLS), OSI model, Browser rendering, DOM, Data Structures, Big O notation

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Sat Jul 18 How Computers Work + The Internet

CONCEPTS

- What is a CPU? (the brain — does all calculations)
- What is RAM? (short-term memory — programs run here)
- What is Storage? (long-term memory — files, databases)
- What is an OS? (Windows/Linux/macOS — manages hardware)
- Process vs Thread; Binary (why 0s and 1s)
- What is the Internet? Physical cables, data centers
- IP Addresses: every device has one (like a house address)
- DNS: the internet's phone book (google.com → 142.250.x.x)
- HTTP/HTTPS: language of the web (request → response)
- SSL/TLS: why the "S" matters

READ / WATCH

CS50x Week 0 (YouTube) — number systems; Fireship "How Computers Work" (8 min); "How the Internet Works in 5 Minutes" (YouTube); ByteByteGo "DNS in 100 Seconds"; MDN "HTTP overview".

HOW TO PRACTICE

Open DevTools → Network tab → visit any website → observe the requests. Convert 5 decimal numbers to binary. Write step-by-step what happens when you type `google.com` and hit Enter (DNS → TCP → HTTPS → HTML → render).

MINI CODE SNIPPET

```
// Decimal → Binary (Feynman practice)
function toBinary(n) {
  const bits = [];
  while (n > 0) { bits.unshift(n % 2); n = n >> 1; }
  return bits.join('') || '0';
}
toBinary(42); // "101010"
```

CONCEPTS

- Full pipeline: URL → DNS → TCP → TLS → HTTP → HTML → DOM → CSS → JS → Page rendered
- What is the DOM? (tree of HTML elements JS can control)
- OSI Model: 7 layers — awareness only
- TCP vs UDP: reliable vs fast (download = TCP, video call = UDP)
- Ports: 80=HTTP, 443=HTTPS, 3000=your local app
- Data structures: Arrays, Objects/Maps, Stack (LIFO), Queue (FIFO)
- Big O: $O(1)$, $O(n)$, $O(n^2)$ — "how slow does code get as data grows?"

READ / WATCH

Fireship "Big O in 100 Seconds"; MDN "Introduction to the DOM"; "Browser Rendering Pipeline" — Google Web Fundamentals; CS Dojo "Data Structures" (YouTube).

HOW TO PRACTICE

Implement Stack and Queue classes in plain JS. Write Big O for each method. Draw the browser rendering pipeline from memory. Explain the OSI model to yourself (Feynman test).

MINI CODE SNIPPET

```
// Stack + Queue in plain JS
class Stack { push(v){ (this.a?=[]).push(v); }
  pop(){ return this.a.pop(); } } // O(1)
class Queue { enq(v){ (this.a?=[]).push(v); }
  deq(){ return this.a.shift(); } } // deq O(n)
```

YOUTUBE + DOCS

- YT** Harvard CS50x Week 0 (binary, hex, memory): <https://youtu.be/y62zj9ozP0M>
- YT** Fireship "How the Internet Works": <https://youtu.be/x3c1ih2NJEg>
- YT** Big O Notation — freeCodeCamp: <https://youtu.be/Mo4vesaut8g>
- YT** Data Structures Easy to Advanced — freeCodeCamp: <https://youtu.be/RBSGKLAvoiM>
- DOCS** MDN "How the Web Works": https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works

Recommended certification: Harvard CS50x on edX (free, verified certificate ~\$149 optional). Also good: freeCodeCamp "Scientific Computing with Python" for algorithmic thinking practice.

PRACTICE QUESTIONS

Q1 —

Implement a Stack class in JS with push, pop, peek, isEmpty. Use it to check balanced parentheses.

Q2 —

Write a function to find duplicates in an array using a hash table (object). $O(n)$ time.

Q3 —

List all OSI layers and give one real-world protocol example per layer.

Q4 —

What is the difference between TCP and UDP? When would you choose UDP over TCP?

DELIVERABLES

- ☐ Binary/hex conversion table in notes
- ☐ "Browser to server" flow diagram drawn
- ☐ Stack, Queue, Hash Map implemented in JS (pushed to GitHub)
- ☐ Notes doc: one paragraph per major concept

MINI PROJECT IDEAS — PHASE 3

1. **DS Toybox** — Vanilla JS classes for Stack, Queue, HashMap, LinkedList, plus Big-O comment for every method. Push to GitHub with README.
2. **Browser Trace** — One-page Markdown "What happens when I type google.com" — DNS → TCP → TLS → HTTP → render pipeline. Add a hand-drawn diagram.
3. **Number System Playground** — HTML+JS tool that converts between decimal, binary, hex, octal. Explain the algorithm on hover.

JS Reset / Revision Week — Jul 20-26, 2026

A one-week catch-up buffer (Mon-Sun) before JavaScript. Consolidate Phases 1-3, warm up JS, then restart JS fresh on Jul 27.

Why this week exists

The JavaScript block slipped by a week. Rather than cram it or carry the gap silently, this is a deliberate **reset week** — it re-locks everything from Phases 1-3 into memory and gets your hands warm on JS syntax, so Phase 4 (Jul 27) starts from strength instead of from rust. It is *revision*, not new theory — low pressure by design.

WHAT TO DO THIS WEEK

REVISE PHASES 1-3 (MORNINGS)

- Redo your Prompt Playbook — tighten 3 weakest prompts
- Re-run your Claude Code slash commands; skim CLAUDE.md
- Rebuild Stack + Queue + HashMap from memory (no notes)
- Re-explain DNS → TCP → HTTPS → render out loud (Feynman)

WARM UP JS (MORNINGS)

- 50 tiny snippets: `let/const`, template literals, ternary
- Rewrite 5 `for` loops as `.map / .filter / .reduce`
- DOM sandbox: build a 20-line vanilla counter + list
- Skim javascript.info "Fundamentals" as a refresher, not a first read

GET AHEAD ON INSTITUTE RESEARCH (EVENINGS)

- Shortlist 2-3 ABAP institutes (Future Labs, Peers, Genius Corner)
- Note fees, batch timings (must fit 12–9 PM office), trainer credentials
- Ask on r/SAP + LinkedIn SAP groups for honest reviews
- Doing this now means Sep 15-17 is *only* enrolment, not research

REST & RESET (SUNDAY JUL 26)

- One full break day — no laptop
- Update the daily organiser app with the new schedule
- Sleep 8 hrs — the 9-day Node phase is coming
- Start Phase 4 on Mon Jul 27 rested, not behind

RESET RULE

This week is a one-time correction, not a habit. If you finish revision early, get ahead on the Phase 4-5 JS reading — do **not** start pulling later phases forward, or you'll erode the breathing room the rest of the plan depends on. One reset, then hold the line.

JavaScript Basics Warm-up

2 days • Jul 27 (Mon) - Jul 28 (Tue) • Pre-workout before Advanced JS

What is this? JavaScript is the language of the web and the foundation of SAP Fiori. These 2 days cover the core syntax — the building blocks used in every JS program you'll write.

Why learn this first? You know some JS from WordPress/Fiori work, but this solidifies the foundation before Advanced JS (Phase 5). Think of it as a quick review + gap-filling.

Topics: let/const/var, data types, type coercion, operators, control flow, functions, arrays, objects, destructuring, spread/rest, DOM basics

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Mon Jul 27 Variables, Types, Functions

CONCEPTS

- var vs let vs const — why var is dangerous (hoisting, no block scope)
- Data types: string, number, boolean, null, undefined, symbol, bigint
- Type coercion: why `1 + "2" = "12"` but `1 - "2" = -1`
- `=` vs `===` (always use `===`)
- Arrow functions vs regular functions (no own `this`)
- Template literals: ``Hello ${name}``; default + rest params
- Control flow: if/else, switch, ternary, optional chaining (`?.`), nullish (`??`)

READ / WATCH

"JavaScript Crash Course" — Traversy Media (first 45 min); MDN "JavaScript data types and data structures"; javascript.info "Data types" chapter.

HOW TO PRACTICE

Write 15 small code snippets: variable scoping, type coercion gotchas, arrow vs regular functions, template literals, optional chaining examples. Push to GitHub.

MINI CODE SNIPPET

```
// Type coercion gotchas
1 + "2"      // "12"    (string concat)
1 - "2"      // -1      (numeric)
[] = ![]     // true    (avoid ==)

const greet = (name = 'friend') => `Hi ${name}`;
greet();     // "Hi friend"
```

DAY 2 Tue Jul 28 Arrays, Objects, Destructuring, DOM

CONCEPTS

- Arrays: create, access, modify. Methods: `.push` `.pop` `.splice` `.slice`
- Higher-order methods: `.map` `.filter` `.find` `.some` `.every` `.forEach`

- Objects: `{key: value}` , accessing properties, methods in objects
- Destructuring: `const {name, age} = user` / `const [a, b] = arr`
- Spread/Rest: `const copy = {...original, newProp: 'val'}`
- DOM: Document Object Model — tree of HTML elements
- Selecting: `getElementById` , `querySelector` ; events: `addEventListener` , `event.target` , `event.preventDefault`

READ / WATCH

"JavaScript DOM Crash Course" — Traversy Media (YouTube); MDN "Destructuring assignment"; javascript.info "Arrays" and "Objects" chapters.

HOW TO PRACTICE

Build a simple DOM to-do list: add, remove, toggle-complete using vanilla JS. No frameworks. Practice all 5 higher-order array methods on real data (filter students by score, etc.).

MINI CODE SNIPPET

```
// Higher-order + destructuring
const users = [{n:'A',s:80},{n:'B',s:60}];
const top = users
  .filter(u => u.s >= 70)
  .map(({n}) => n); // ['A']

// Swap without temp
let [a, b] = [1, 2]; [a, b] = [b, a];
```

YOUTUBE + DOCS

YT Traversy "JavaScript Crash Course For Beginners": <https://youtu.be/hdI2bq0jy3c>

YT Traversy "JavaScript DOM Crash Course" (full playlist): <https://youtu.be/0ik6X4DJKCc>

DOCS javascript.info (best free JS book): <https://javascript.info/>

DOCS MDN JS Guide: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

Recommended certification: freeCodeCamp "JavaScript Algorithms and Data Structures" (free certificate, ~300 hrs of self-paced practice — do the first 60 hrs now, continue in evenings).

PRACTICE QUESTIONS

Q1 —

What does `[] + {}` evaluate to in JS? Explain why (type coercion).

Q2 —

Write a function that takes an array of objects and returns only items where `active === true` , sorted by name.

Q3 —

Use destructuring to swap two variables without a temp variable.

Q4 —

Build a `querySelector` -based counter: two buttons (+ and -), display updates live in the DOM.

DELIVERABLES

- [] 15 code snippets file pushed to GitHub
- [] DOM to-do list mini app complete
- [] All 4 practice questions answered

MINI PROJECT IDEAS — PHASE 4

1. **Vanilla To-Do** — Zero-framework to-do list with add / edit / delete / mark-complete + localStorage persistence. Practise all higher-order array methods.
2. **Type-Coercion Quiz** — Quiz app with 15 "what does this JS output?" questions (trickies like `[] + {}`). Score + explanation reveal.
3. **DOM Counter Pro** — Increment/decrement counter with step selector, keyboard shortcuts, undo/redo — all vanilla, no React.

Advanced JavaScript + OOP

4 days • Jul 29 (Wed) - Aug 1 (Sat) • The most important JS phase

What is this? Deep-dive into how JavaScript actually works under the hood — the runtime, scope, closures, prototypes, classes, event loop, and async patterns.

Why should you learn this? Every React hook, every Node.js callback, every Fiori controller, every Claude API tool call — they all sit on these fundamentals. Interview questions like "explain closures" or "how does the event loop work" are asked at every level. Miss this and Phase 6 (React) will feel like magic instead of code.

Topics covered: Scope + hoisting + closures, `this` binding + `call`/`apply`/`bind`, prototypes & prototype chain, ES6 classes, inheritance, encapsulation (`# private` fields), polymorphism, getters/setters, static methods, event loop + microtasks, Promises, `async/await`, `.map`/`.filter`/`.reduce`, ES modules

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Wed Jul 29 Scope, Hoisting, Closures

CONCEPTS

- Execution Context (Global + Function scope)
- Hoisting: `var` to top; `let/const` in Temporal Dead Zone
- Lexical Scope: functions see variables from where they were *written*
- Closure = function + its outer variables ("remembers its birthplace")
- IIFE (Immediately Invoked Function Expression)

READ / WATCH

Namaste JavaScript Ep 1-6 (Akshay Saini): <https://youtu.be/pN6jk0uUrD8> .

HOW TO PRACTICE

Build: (1) counter with closure — private count, no globals. (2) `memoize(fn)` using closure. (3) Module pattern with IIFE.

MINI CODE SNIPPET

```
// Counter via closure (no globals)
function counter() {
  let count = 0;
  return {
    inc: () => ++count,
    dec: () => --count,
    val: () => count,
  };
}
const c = counter();
c.inc(); c.inc(); c.val(); // 2
```

DAY 2 Thu Jul 30 OOP Part 1 — Prototypes + Classes

CONCEPTS

- OOP 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction
- Every JS value has a hidden `[[Prototype]]` link
- Prototype chain: how `obj.toString()` is found on `Object.prototype`
- Constructor functions (old way): `function User(name){this.name=name}` + `new`
- ES6 `class` : constructor, methods, static, getters/setters
- `this` binding: default, implicit, explicit (call/apply/bind), new
- Arrow functions have **no own** `this`

READ / WATCH

Namaste JS Ep 7–9; MDN "Inheritance and the prototype chain".

HOW TO PRACTICE

Write a `Vehicle` constructor, add methods on prototype, create `Car` + `Bike` instances. Rewrite via ES6 `class` .
Write all 4 binding rules of `this` .

MINI CODE SNIPPET

```
class Vehicle {
  constructor(kind) { this.kind = kind; }
  describe() { return `I am a ${this.kind}`; }
}
class Car extends Vehicle {
  constructor() { super('car'); }
}
new Car().describe(); // "I am a car"
```

DAY 3 Fri Jul 31 OOP Part 2 — Inheritance, Encapsulation, Polymorphism

CONCEPTS

- Inheritance: `class Car extends Vehicle` , using `super()`
- # private fields (2022+): `#password` is truly private
- Getters/setters: `get fullName()` , `set fullName(v)`
- Polymorphism: same method name, different behavior (override in subclass)
- Abstraction: hide implementation behind a public method
- Static methods vs instance methods
- `Object.create` , `Object.assign` , spread on classes

READ / WATCH

Web Dev Simplified "OOP in JS"; javascript.info "Classes" chapter.

HOW TO PRACTICE

Build: `Shape` base → `Circle` , `Rectangle` extend → each overrides `area()` . Add # private radius. Static factory `Shape.random()` . Show polymorphism via `shapes.forEach(s ⇒ s.area())` .

MINI CODE SNIPPET

```
class Shape { area(){ throw 'override'; } }
class Circle extends Shape {
  #r; // # private field
  constructor(r){ super(); this.#r = r; }
  area(){ return Math.PI * this.#r ** 2; }
}
[new Circle(3)].forEach(s => console.log(s.area()));
```

DAY 4 Sat Aug 1 Event Loop + Async/Await + Modules

CONCEPTS

- JS is single-threaded but async: Call Stack + Web APIs + Task Queue + Microtask Queue
- Microtasks (Promise.then) run *before* next macrotask (setTimeout)
- Callbacks → Callback Hell → Promises → async/await evolution
- Promise states: pending / fulfilled / rejected. .then/.catch/.finally
- `async` always returns a Promise; `await` pauses only that async function
- Promise.all / .allSettled / .race
- ES Modules: `export` / `import`, default vs named
- Mastery of `.map` / `.filter` / `.reduce`

READ / WATCH

Philip Roberts "What the heck is the event loop?" JSConf EU: <https://youtu.be/8aGhZQkoFbQ>; Namaste JS Ep 10–14.

HOW TO PRACTICE

Build async TaskQueue class (mini project). Build `Promise.allSettled` from scratch. Predict output of tricky event-loop code. Split code into ES modules.

MINI CODE SNIPPET

```
// Predict the output
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
// Order: 1, 4, 3, 2 (microtasks before macrotasks)
```

YOUTUBE + DOCS

YT Namaste JavaScript — Akshay Saini (playlist, must-watch): <https://youtu.be/pN6jk0uUrD8>

YT Web Dev Simplified "OOP in JavaScript": <https://youtu.be/PFmuCDHHpwk>

YT "What the heck is the event loop" — Philip Roberts: <https://youtu.be/8aGhZQkoFbQ>

YT Fireship "Async Await" (7 min): <https://youtu.be/vn3tm0quoqE>

DOCS javascript.info Classes: <https://javascript.info/classes>

Recommended certifications: (1) freeCodeCamp "JavaScript Algorithms and Data Structures" — free cert. (2) HackerRank "JavaScript" Basic + Intermediate badges (30 min each, adds to resume). (3) Meta "Programming with JavaScript" on Coursera (free audit, paid cert).

PRACTICE QUESTIONS

Q1 —

Write a `counter()` closure that returns `{ increment, decrement, reset, value }`. Each call to increment/decrement should update the private count.

Q2 —

Implement `debounce(fn, delay)` using closures and `setTimeout`. Test it by attaching to a search input.

Q3 —

Write `Promise.allSettled` from scratch without using the built-in version.

Q4 —

Using only `.reduce()`, write a function that takes an array of words and returns an object of word frequencies.

Q5 —

Explain: what is the output of this code and why? `console.log(1);`
`setTimeout(()⇒console.log(2),0); Promise.resolve().then(()⇒console.log(3));`
`console.log(4);`

MINI PROJECT

Build an async task queue: `TaskQueue` class that runs async tasks one at a time (concurrency: 1), with `add(fn)` method, automatic processing, and handles errors gracefully. Uses closures, Promises, and class syntax.

DELIVERABLES

- [] Closure examples: counter, memoize, module pattern
- [] Prototype + class inheritance examples
- [] Callback → Promise → async/await rewrite
- [] `Promise.all` built from scratch
- [] `TaskQueue` mini project on GitHub
- [] All 5 practice questions answered

MINI PROJECT IDEAS — PHASE 5

1. **TaskQueue Class** — Promise-based queue that runs async tasks one-at-a-time (concurrency:1), with `add(fn)`, error isolation, and pause/resume.
2. **Rate-Limited Fetcher** — Wrapper around `fetch` that debounces (300ms) and rate-limits (max 5 concurrent). Use closures + Promises.
3. **Mini Redux** — Tiny state store in ~30 lines: `createStore(reducer)` returning `{ getState, dispatch, subscribe }`. Prove it works with a counter.

Break Day — Aug 2 (Sunday)

Purpose: Revise and consolidate Phase 5 Advanced JS + OOP before starting React.

- Re-read your Phase 5 notes from Day 1-4 — especially OOP
- Redo the Shape class + Promise.allSettled from memory without looking
- Watch the Philip Roberts event-loop talk again if anything felt unclear
- Push all Phase 5 code to GitHub
- Prepare React environment: `npm create vite@latest my-react-app -- --template react`, install Tailwind
- Rest in the afternoon. Phase 6 starts fresh Monday Aug 3.

React + Tailwind CSS

6 days • Aug 3 (Mon) - Aug 8 (Sat)

What is this? React is the world's most-used UI library and the same technology behind SAP Fiori (which uses SAPUI5, a similar MVC pattern). Tailwind is a utility-first CSS framework. Together they let you ship modern, professional UIs fast.

Why should you learn this? (1) Your capstone FocusTrack Pro uses React. (2) SAP Fiori concepts (components, data binding, routing) map directly to React — learning React makes UI5 easier later. (3) Every job posting for SAP + Web asks for React or UI5.

Topics covered: JSX, components, props, useState, useEffect, forms, React Router v6, useContext, useReducer, custom hooks (useFetch, useLocalStorage, useDebounce), Tailwind utility classes, responsive design

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Mon Aug 3 JSX + Components + Props

CONCEPTS

- React = library for building UIs from small reusable pieces (*components*)
- JSX = "HTML in JavaScript": `<div>Hi {name}</div>`
- Vite setup: `npm create vite@latest , folder tour`
- Function components only: `function Button(props){...}`
- Props = arguments; read-only
- Conditional rendering: `{isLoggedIn ? <Dashboard/> : <Login/>}`
- Lists with `.map` + unique `key` prop

READ / WATCH

React docs "Quick Start": <https://react.dev/learn> ; Net Ninja React Tutorial: <https://youtu.be/j942wKiXFu8> .

HOW TO PRACTICE

Create 5 components: Button, Card, Badge, Avatar, Alert. Pass props, render conditionally, render arrays with keys.

MINI CODE SNIPPET

```
export function Badge({ text, tone = 'info' }) {
  const tones = {
    info: 'bg-blue-100 text-blue-700',
    warn: 'bg-amber-100 text-amber-700',
  };
  return <span className={`px-2 py-1 rounded ${tones[tone]} `>
    {text}
  </span>;
}
```

DAY 2 Tue Aug 4 useState + useEffect + Forms

CONCEPTS

- State = data that changes over time: `const [count, setCount] = useState(0)`
- `useEffect` = "run this side-effect after render" (data fetch, subscriptions)
- Cleanup functions (return a function from `useEffect`)
- Controlled inputs: `value` + `onChange`
- Handling loading / error / success states

READ / WATCH

React docs "Adding Interactivity"; Web Dev Simplified `useState`: <https://youtu.be/06P86uwfdR0> .

HOW TO PRACTICE

Build a controlled form (name, email, select). Fetch users from

<https://jsonplaceholder.typicode.com/users> with `useEffect`, show loading/error/success.

MINI CODE SNIPPET

```
function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch('/api/users').then(r => r.json())
      .then(setUsers).finally(() => setLoading(false));
  }, []);
  if (loading) return <p>Loading...</p>;
  return users.map(u => <li key={u.id}>{u.name}</li>);
}
```

DAY 3 Wed Aug 5 React Router v6

CONCEPTS

- Client-side routing without a full page reload
- `<BrowserRouter>` , `<Routes>` , `<Route path element>`
- Dynamic params: `/posts/:id` + `useParams()`
- `<Link>` and `useNavigate()` for programmatic navigation
- Nested routes, layout routes, 404 wildcard `*`

READ / WATCH

React Router v6 tutorial: <https://reactrouter.com/en/main/start/tutorial> ; Traversy React Router.

HOW TO PRACTICE

Build a 3-page app: Home, Posts list, Post detail (`useParams`). Add a 404 wildcard. Use `useNavigate` on a "Back" button.

MINI CODE SNIPPET


```

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/posts/:id" element={<Post />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
</BrowserRouter>
// In Post: const { id } = useParams();

```

DAY 4 Thu Aug 6 useContext + Custom Hooks

CONCEPTS

- Prop drilling → Context solves it
- `createContext` + `Provider` + `useContext`
- Custom hook = a function starting with `use` that uses other hooks
- Extract repeated logic into: `useFetch`, `useLocalStorage`, `useDebounce`

READ / WATCH

React docs "Passing Data Deeply with Context"; Web Dev Simplified Custom Hooks.

HOW TO PRACTICE

Build: ThemeContext (light/dark); AuthContext (mock login); custom hooks `useFetch`, `useLocalStorage`, `useDebounce`.

MINI CODE SNIPPET

```

export function useDebounce(value, delay = 300) {
  const [v, setV] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setV(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return v;
}

```

DAY 5 Fri Aug 7 useReducer + Tailwind CSS

CONCEPTS

- `useReducer` = `useState` for complex state (reducer + dispatch)
- When to prefer `useReducer` over `useState`
- Tailwind utility classes: `<div class="p-4 rounded bg-blue-500">`
- Responsive: `md:grid-cols-3`. Dark: `dark:bg-slate-900`. Hover: `hover:bg-blue-600`

READ / WATCH

React `useReducer` docs; Tailwind "Core Concepts"; Dave Gray Tailwind: <https://youtu.be/LCxcTs0Hrjo>.

HOW TO PRACTICE

Rewrite Day 2 form using `useReducer`. Style 5 components using only Tailwind classes (no custom CSS). Build a responsive card grid.

MINI CODE SNIPPET

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'ADD': return [...state, action.item];
    case 'REMOVE': return state.filter(i => i.id !== action.id);
    default: return state;
  }
};
const [items, dispatch] = useReducer(reducer, []);
```

DAY 6 Sat Aug 8 Mini Project Day — Task Tracker App

CONCEPTS

- Combine every concept from Days 1–5
- Use weekend time (9–2 study + 6–9 project block) to finish it

READ / WATCH

React perf docs: `useMemo`, `useCallback` (read-only today).

HOW TO PRACTICE

Build Task Tracker: add/edit/delete tasks, filter by status (All/Active/Done), persist to localStorage, light/dark theme via Context + Tailwind. Deploy on Vercel.

MINI CODE SNIPPET

```
// Tailwind responsive card grid
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {tasks.map(t => (
    <article key={t.id}
      className="p-4 rounded-lg bg-white shadow hover:shadow-md dark:bg-slate-800">
        <h3 className="font-semibold">{t.title}</h3>
      </article>
    )
  )}
</div>
```

YOUTUBE + DOCS

YT Net Ninja React 18 (full playlist): <https://youtu.be/j942wKixFu8>

YT Dave Gray Tailwind CSS Full Course: <https://youtu.be/lCxcTs0Hrjo>

YT Traversy React Router v6: <https://youtu.be/oTIJunBa6MA>

YT Web Dev Simplified Custom Hooks: <https://youtu.be/6ThXsUwLWvc>

DOCS React docs: <https://react.dev> | Tailwind docs: <https://tailwindcss.com/docs>

Recommended certifications: (1) Meta "React Basics" on Coursera (free audit, paid cert). (2) freeCodeCamp "Front End Development Libraries" — free cert. (3) HackerRank "React (Basic)" badge.

PRACTICE QUESTIONS

Q1—

Build a controlled search input that debounces API calls by 500ms using a custom `useDebounce` hook.

Q2 —

Create a `useLocalStorage(key, initialValue)` custom hook that syncs state to localStorage automatically.

Q3 —

Implement a shopping cart with useReducer: actions ADD_ITEM, REMOVE_ITEM, UPDATE_QTY, CLEAR_CART.

Q4 —

Create a protected route component that redirects unauthenticated users to /login using React Router v6.

Q5 —

What is the difference between useEffect cleanup function and componentWillUnmount? When does cleanup run?

MINI PROJECT — TASK TRACKER

- Add / edit / delete tasks
- Filter by status (All / Active / Done)
- Persist to localStorage (useLocalStorage hook)
- Light / dark theme (Context + Tailwind)
- React Router: /tasks, /tasks/:id
- Deployed live URL in README

DELIVERABLES

- [] 5 UI components built (Day 1)
- [] API fetch with loading/error states (Day 2)
- [] 3-page Router app (Day 3)
- [] ThemeContext + AuthContext (Day 4)
- [] useReducer form + Tailwind grid (Day 5)
- [] Task Tracker deployed (Day 6)

MINI PROJECT IDEAS — PHASE 6

1. **Task Tracker App (Vercel)** — React + Tailwind + Router + Context + custom hooks + localStorage. Add / edit / delete / filter (All/Active/Done). Deploy to Vercel with live URL.
2. **Weather Dashboard** — Fetches from a free weather API using a `useFetch` hook; supports multiple saved cities via Context; dark mode toggle in Tailwind.
3. **Component Library Sketch** — 5 reusable UI components (Button, Card, Modal, Toast, Dropdown) styled with Tailwind + a small Storybook-style demo page.

Break Day — Aug 9 (Sunday)

Purpose: Revise React + Tailwind before the long Node.js phase.

- Review your custom hooks: `useFetch`, `useLocalStorage`, `useDebounce` — can you write each from memory?
- Polish Task Tracker app — fix rough edges, update README with screenshots
- Set up Node.js project folder: `mkdir node-api && cd node-api && npm init -y`
- Phase 7 starts fresh Monday Aug 10.

Node.js + Express (Deep Dive)

9 days • Aug 10 (Mon) - Aug 18 (Tue)

What is this? Node.js runs JavaScript on the server. Express is the most-used web framework on top of Node. Together they let you build REST APIs, auth systems, file uploads, and everything a backend needs.

Why should you learn this? (1) Your capstone backend runs on Express. (2) Even as an SAP engineer, side scripts / integration jobs / BTP CAP handlers are often Node.js. (3) 9 days of deep practice = you can build any backend from memory by the end.

Topics covered: Node runtime + npm + fs/path/http, Express setup + routing, req/res deep-dive, middleware pipeline (built-in, custom, cors/helmet/morgan), express.Router, async error handling, MVC structure, .env, bcrypt password hashing, JWT auth, zod validation, rate limiting, multer file uploads

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Mon Aug 10 Node Runtime + npm + Built-in Modules

CONCEPTS

- Node = V8 engine + libuv (async I/O) outside the browser
- npm basics: `npm init`, `package.json`, dependencies vs devDependencies
- Built-ins: `fs`, `path`, `http`
- Sync vs Async: never use `fs.readFileSync` in prod — use `fs.promises`

READ / WATCH

Node docs Getting Started; Traversy Node Crash Course: <https://youtu.be/fBNz5xF-Kx4>.

HOW TO PRACTICE

Read/write files with `fs.promises`. Build a raw HTTP server with `http.createServer`, respond with JSON.

MINI CODE SNIPPET

```
// fs.promises + async
import { readFile, writeFile } from 'node:fs/promises';
const data = await readFile('config.json', 'utf8');
const cfg = JSON.parse(data);
await writeFile('out.json', JSON.stringify(cfg, null, 2));
```

DAY 2 Tue Aug 11 Express Setup + req/res

CONCEPTS

- Why Express: routing/middleware trivial vs raw `http`
- `const app = express()`, `app.get(path, handler)`
- `req.params` vs `req.query` vs `req.body`
- `req.headers`, `req.cookies`
- `res.json()`, `res.status()`, `res.redirect()`

READ / WATCH

HOW TO PRACTICE

Build an Express app with 5 routes demonstrating `req.params`, `req.query`, `req.body` (JSON POST), `req.headers`, and all `res` methods.

MINI CODE SNIPPET

```
import express from 'express';
const app = express();
app.use(express.json());

app.get('/users/:id', (req, res) => {
  res.status(200).json({ id: req.params.id });
});
app.listen(3000);
```

DAY 3 Wed Aug 12 Middleware Pipeline

CONCEPTS

- Middleware = `function(req, res, next)` between request and handler
- Order matters — runs top to bottom
- Built-in: `express.json()`, `express.static()`
- Third-party: cors, helmet, morgan (logging)
- Custom: logger, timer, API key check

READ / WATCH

Express "Writing Middleware" docs; Web Dev Simplified Middleware.

HOW TO PRACTICE

Write 3 custom middleware (request logger, timer, API key auth). Add cors + helmet + morgan.

MINI CODE SNIPPET

```
// Custom request timer middleware
const timer = (req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    console.log(`${req.method} ${req.url} - ${Date.now() - start}ms`);
  });
  next();
};
app.use(timer);
```

DAY 4 Thu Aug 13 Router + Async Error Handling

CONCEPTS

- `express.Router()` to split routes into modules
- Static files: `app.use(express.static('public'))`
- Async errors don't propagate to error middleware — wrap with `asyncHandler` or `try/catch`
- Custom error class: `class AppError extends Error`

READ / WATCH

Express Routing + Error Handling docs; `express-async-handler` npm.

HOW TO PRACTICE

Split routes into `authRouter` + `usersRouter` . Serve `/public` . Wrap all async handlers with `asyncHandler` .

MINI CODE SNIPPET

```
// asyncHandler + custom error class
const asyncHandler = fn => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

class AppError extends Error {
  constructor(msg, status = 500) {
    super(msg); this.status = status;
  }
}
```

DAY 5 Fri Aug 14 MVC + Global Error Middleware

CONCEPTS

- MVC (for APIs): Controllers + Services + Repositories
- Folder structure: `routes/` , `controllers/` , `services/` , `models/`
- Global error middleware: 4 args (`err`, `req`, `res`, `next`)
- Standard shape: `{ success:false, message, statusCode }`

READ / WATCH

Traversy Node REST API MVC section; Net Ninja MVC Express.

HOW TO PRACTICE

Refactor routes into MVC. Write global error handler that returns standard JSON.

MINI CODE SNIPPET

```
// Global error handler (4 args)
app.use((err, req, res, next) => {
  const status = err.status || 500;
  res.status(status).json({
    success: false,
    message: err.message,
    statusCode: status,
  });
});
```

DAY 6 Sat Aug 15 .env + bcrypt + JWT Auth

CONCEPTS

- dotenv: read `.env` , never commit secrets
- bcrypt: hash passwords with salt (never store plaintext)
- JWT = JSON Web Token — signed payload, server verifies without DB
- Access (short-lived) + refresh (long-lived) tokens
- Auth middleware: read Bearer, verify, attach `req.user`

READ / WATCH

Traversy JWT Auth: <https://youtu.be/mbsmsi7l3r4> ; `bcrypt` + `jsonwebtoken` npm docs.

HOW TO PRACTICE

Implement register (bcrypt hash), login (compare hash, issue JWT), `protect` middleware. Test all with VS Code REST Client.

MINI CODE SNIPPET

```
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';

const hash = await bcrypt.hash(password, 10);
const ok = await bcrypt.compare(input, hash);

const token = jwt.sign({ userId }, process.env.JWT_SECRET,
  { expiresIn: '15m' });
```

DAY 7 Sun Aug 16 zod Validation + Rate Limiting

CONCEPTS

- Validate at boundary — trust nothing from client
- zod: strings, numbers, enums, arrays, nested objects, refinements
- Middleware: `parse req.body` with schema, return 422 with field errors
- Rate limiting: `express-rate-limit` — 100 req/15min global, 5/15min on login

READ / WATCH

zod docs: <https://zod.dev> ; Matt Pocock zod tutorial.

HOW TO PRACTICE

Write zod schemas for register, login, product create. Add validation middleware. Add rate limiting.

MINI CODE SNIPPET

```
import { z } from 'zod';

const RegisterSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
  role: z.enum(['user', 'admin']).default('user'),
});
const body = RegisterSchema.parse(req.body);
```

DAY 8 Mon Aug 17 File Uploads + CORS

CONCEPTS

- `multer` for multipart/form-data uploads
- `diskStorage` vs `memoryStorage`; filename sanitization
- Limits: `fileSize`, files count; `fileFilter` for MIME check
- CORS whitelist: `cors({ origin: ['https://app.com'] })`

READ / WATCH

multer npm docs (storage, limits, fileFilter).

HOW TO PRACTICE

Implement single + multiple upload with type filter (jpg/png/pdf). Configure CORS whitelist.

MINI CODE SNIPPET

```
import multer from 'multer';
const upload = multer({
  storage: multer.diskStorage({ destination: 'uploads/' }),
  limits: { fileSize: 2 * 1024 * 1024 },
  fileFilter: (_, file, cb) =>
    cb(null, /^image\/.test(file.mimetype)),
});
app.post('/avatar', upload.single('file'), handler);
```

DAY 9 Tue Aug 18 Full App Integration + Mini Project

CONCEPTS

- Put everything together into the Auth API
- Test end-to-end with REST Client / Postman
- Deliverable: working, tested, pushed to GitHub

READ / WATCH

Review Days 1–8. "Node.js Best Practices" GitHub repo (top 10 only).

HOW TO PRACTICE

Build complete Auth API: register + login + get profile + update profile with avatar upload. Full MVC + JWT + zod + multer. Push to GitHub with README.

MINI CODE SNIPPET

```
// Suggested folder tree
src/
├─ routes/      auth.routes.js users.routes.js
├─ controllers/ auth.controller.js
├─ services/    auth.service.js
├─ repositories/ users.repo.js
├─ middleware/  auth.js validate.js error.js
├─ config/      env.js db.js
└─ app.js
```

YOUTUBE + DOCS

YT Traversy Node Crash Course: <https://youtu.be/fBNz5xF-Kx4>

YT Traversy Express Crash Course: <https://youtu.be/L72fhGm1tfE>

YT Traversy Node JWT Auth: <https://youtu.be/mbsmsi7L3r4>

YT Net Ninja Node REST + MVC (playlist): <https://youtu.be/tisSgWQeQ8s>

DOCS Express: <https://expressjs.com/> | Zod: <https://zod.dev> | Node best practices: <https://github.com/goldbergonyi/nodebestpractices>

Recommended certifications: (1) freeCodeCamp "Back End Development and APIs" — free cert. (2) OpenJS Node.js Application Developer (JSNAD) — official cert (paid, but very respected).

PRACTICE QUESTIONS

Q1 —

Write an async middleware `requestTimer` that logs `METHOD /path - Xms` for every request using `Date.now()`.

Q2 —

Implement a JWT `protect` middleware that: reads token from Authorization header, verifies it, attaches `req.user`, returns 401 if missing/invalid.

Q3 —

Write a zod schema for product creation: name (string, min 3), price (positive number), category (enum: electronics/clothing/food), images (array of strings, max 5).

Q4 —

Write a global Express error handler that: handles zod validation errors (422), JWT errors (401), and unknown errors (500) with different response shapes.

Q5 —

Implement multer `fileFilter` that rejects files larger than 2MB or with non-image MIME types, throwing a custom error caught by global error handler.

MINI PROJECT — AUTH REST API

- POST `/auth/register` — zod validate, hash password, return JWT
- POST `/auth/login` — compare hash, issue JWT + refresh token
- GET `/users/me` — protected, return user profile
- PUT `/users/me` — protected, validate with zod, upload avatar with multer
- Full MVC: routes / controllers / services
- Global error handler + `asyncHandler` on all routes
- Rate limiting on `/auth` routes

DELIVERABLES CHECKLIST

- ☐ Raw HTTP server built (Day 1)
- ☐ Express app with all req/res types (Day 2)
- ☐ 3 custom middleware + cors/helmet (Day 3)
- ☐ Router split + `asyncHandler` (Day 4)
- ☐ MVC refactor + global error handler (Day 5)
- ☐ JWT auth working end-to-end (Day 6)
- ☐ zod validation + rate limiting (Day 7)
- ☐ multer file upload (Day 8)
- ☐ Full Auth API on GitHub (Day 9)

MINI PROJECT IDEAS — PHASE 7

1. **Auth REST API** — POST `/register` + `/login` (bcrypt + JWT), GET/PUT `/users/me` (protected), avatar upload (multer). MVC structure, zod validation, global error handler, rate-limited `/auth`.
2. **URL Shortener** — Express + in-memory Map first, then MySQL later. POST `/shorten` → returns code; GET `/:code` → 302 redirect; simple analytics counter.
3. **File Upload Service** — Standalone Express service: multipart uploads, image type + size validation, saves to disk (later S3), returns signed URL. Great AWS practice.

REST API Design

3 days • Aug 19 (Wed) - Aug 21 (Fri)

What is this? The design discipline of naming URLs, choosing HTTP verbs, returning consistent errors, paginating results, versioning, and documenting APIs so other developers can use them without asking questions.

Why should you learn this? Phase 7 taught you HOW to build an API. Phase 8 teaches you how to design one that doesn't need explaining. SAP OData services, Fiori apps, and CAP services all follow REST-like conventions — getting fluent here helps you read/design SAP APIs later.

Topics covered: Resource URLs, HTTP verbs + idempotency, status codes, standard error format, offset vs cursor pagination, filtering + sorting + search, API versioning, OpenAPI 3.0 / Swagger docs

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Wed Aug 19 Resource URLs + Verbs + Status Codes

CONCEPTS

- URLs = nouns (resources), Verbs = actions
- GET (safe, idempotent), POST (create), PUT (full replace, idempotent), PATCH (partial), DELETE (idempotent)
- Status codes: 2xx success, 3xx redirect, 4xx client error, 5xx server error
- Standard error JSON: `{ error: { code, message, details } }`

READ / WATCH

freeCodeCamp REST Best Practices; Microsoft REST Guidelines: <https://github.com/microsoft/api-guidelines>.

HOW TO PRACTICE

Design blog API URLs (posts, comments, tags, authors). Map every operation to verb + status code. Write standard error response JSON.

MINI CODE SNIPPET

```
# Resource design for a blog API
GET    /api/v1/posts           # 200 list
POST   /api/v1/posts         # 201 created
GET    /api/v1/posts/:id     # 200 / 404
PATCH /api/v1/posts/:id     # 200 partial
DELETE /api/v1/posts/:id     # 204
POST   /api/v1/posts/:id/comments # nested
```

DAY 2 Thu Aug 20 Pagination + Filtering + Sorting

CONCEPTS

- Offset pagination: `?limit=20&offset=40` (simple, slow at deep offsets)
- Cursor pagination: `?limit=20&cursor=abc` (stable under inserts)
- Filtering: `?status=published&author=nikhil`

- Sorting: `?sort=-createdAt,+title`

READ / WATCH

Stripe engineering "Pagination" blog; cursor-based pagination articles.

HOW TO PRACTICE

Implement both offset + cursor pagination for a `/posts` endpoint. Support `?filter[status]` + `?sort=-createdAt` + `?limit`.

MINI CODE SNIPPET

```
// Cursor pagination
GET /posts?limit=20&cursor=eyJpZCI6MTIzfQ==
→ { data: [...], nextCursor: 'eyJpZCI6MTQzfQ==' }

// Encode/decode cursor server-side
const encode = obj ⇒ Buffer.from(JSON.stringify(obj)).toString('base64');
const decode = c ⇒ JSON.parse(Buffer.from(c, 'base64'));
```

DAY 3 Fri Aug 21 Versioning + OpenAPI/Swagger

CONCEPTS

- Versioning options: URL prefix `/v1/`, header, media type. URL is most common
- OpenAPI 3.0 = spec for describing APIs in YAML/JSON
- Swagger UI serves docs from your spec — try-it-out UI
- Contract-first vs code-first approaches

READ / WATCH

Swagger docs: <https://swagger.io/docs/specification/about/>; OpenAPI 3.0 tutorial on YouTube.

HOW TO PRACTICE

Prefix Phase 7 Auth API with `/api/v1/`. Write `openapi.yaml` for all 4 routes. Serve Swagger UI at `/api-docs` with `swagger-ui-express`.

MINI CODE SNIPPET

```
# openapi.yaml (excerpt)
paths:
  /api/v1/auth/register:
    post:
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Register'
      responses:
        '201': { description: Created }
        '422': { description: Validation error }
```

YOUTUBE + DOCS

YT Fireship "REST API Design Best Practices" (8 min): <https://youtu.be/-MTSQjw5DrM>

YT Laith Academy "Swagger UI with Express" on YouTube

DOCS OpenAPI 3.0 spec: <https://swagger.io/specification/> | Microsoft REST guidelines: <https://github.com/microsoft/api-guidelines>

Recommended certifications: Postman "API Fundamentals Student Expert" (free, badge). Also: Google Cloud "Apigee API Design" course (free) for enterprise-scale API patterns.

PRACTICE QUESTIONS

Q1 —

Design the full URL structure for an e-commerce API with: products, categories, orders, order-items, users, reviews. Show all CRUD operations.

Q2 —

What HTTP status codes would you return for: successful creation, not found, validation error, unauthorized, forbidden, server crash?

Q3 —

Implement cursor-based pagination for a messages endpoint where each message has an auto-increment ID. Show the cursor encode/decode logic.

Q4 —

Write an OpenAPI 3.0 YAML spec for POST `/api/v1/auth/register` with requestBody schema (zod-style) and all possible response codes.

MINI PROJECT

Extend the Phase 7 Auth API: add `/api/v1/` prefix, implement offset + cursor pagination for a `/posts` endpoint, add filtering (`?status`, `?author`) and sorting (`?sort`), serve Swagger UI at `/api-docs` with full `openapi.yaml` documentation.

DELIVERABLES

- [] Blog API URL design document
- [] Error format spec written
- [] Offset + cursor pagination implemented
- [] API versioned under `/api/v1/`
- [] `openapi.yaml` written
- [] Swagger UI serving at `/api-docs`

MINI PROJECT IDEAS — PHASE 8

1. **Blog API v1** — Extend Phase 7 Auth API: add posts + comments + tags. Full pagination (offset + cursor), filtering, sorting. Serve Swagger UI at `/api-docs`.
2. **E-commerce Catalog API** — Design + implement products, categories, orders. Include OpenAPI 3.0 `openapi.yaml`. Focus on consistent error format + status codes.
3. **Postman Test Suite** — For any API above, write a Postman collection + environment covering 20 happy-path + edge cases. Export JSON and commit to repo.

Break Days — Aug 22-23 (Saturday + Sunday)

Purpose: Full weekend off — revise Phases 7 + 8 before starting MySQL.

- **Aug 22 (Sat):** Review all Phase 7 Node/Express code. Rebuild the JWT auth middleware from memory in a new file — no peeking.
- **Aug 23 (Sun):** Review Phase 8 REST design. Design a new API from scratch (pick any domain). Write a quick openapi.yaml for it.
- Install MySQL Workbench + XAMPP if not already done. Create a test database. Run a few basic queries to feel comfortable.
- Phase 9 MySQL starts fresh Monday Aug 24.

MySQL + Node.js Integration

7 days • Aug 24 (Mon) - Aug 30 (Sun)

What is this? Relational databases store data in tables with relationships. SQL (Structured Query Language) is how you talk to them. MySQL is the world's most popular open-source RDBMS.

Why should you learn this? (1) SAP runs on databases — understanding relational modeling transfers directly to HANA and ABAP Dictionary. (2) SQL is the same across MySQL, PostgreSQL, and HANA (95%). (3) Your capstone stores data in MySQL. (4) Every backend interview asks JOIN + normalization questions.

Topics covered: DDL (CREATE/ALTER/DROP), DML (INSERT/UPDATE/DELETE), DQL (SELECT with WHERE/ORDER/LIMIT/LIKE), JOINS, GROUP BY + aggregations, subqueries, indexes + EXPLAIN, normalization (1NF-3NF), views, transactions, stored procedures, mysql2/promise driver, connection pooling, Repository pattern

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Mon Aug 24 MySQL Setup + DDL

CONCEPTS

- Tables = rows + columns, like a very strict Excel
- Install MySQL Server + Workbench; `CREATE DATABASE`, `USE database`
- Data types: INT, VARCHAR(n), TEXT, DATE, DATETIME, BOOLEAN, DECIMAL
- `CREATE TABLE` with PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT
- `ALTER TABLE` ADD/DROP/MODIFY; `DROP TABLE`; `TRUNCATE` vs `DELETE`

READ / WATCH

Programming with Mosh MySQL (first hour): https://youtu.be/7S_tz1z_5bA; MySQL Data Types docs.

HOW TO PRACTICE

Create school DB: students, courses, enrollments. Add constraints. Drop and recreate from a script.

MINI CODE SNIPPET

```
-- School schema (3NF)
CREATE TABLE students (
  id      INT PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(100) NOT NULL,
  email   VARCHAR(120) UNIQUE NOT NULL,
  created DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_students_name ON students(name);
```

DAY 2 Tue Aug 25 DML + Basic SELECT

CONCEPTS

- `INSERT INTO ... VALUES` (single + multi-row)
- `UPDATE ... SET ... WHERE` (always WHERE!)

- DELETE FROM ... WHERE
- SELECT columns FROM table WHERE conditions
- ORDER BY (ASC/DESC), LIMIT + OFFSET, LIKE, IN, BETWEEN

READ / WATCH

Mosh MySQL Ep 2–3; MySQL SELECT docs.

HOW TO PRACTICE

Insert 20 rows using multi-row INSERT. Write 10 SELECT queries: multi-condition filter, LIKE search, ORDER BY 2 cols, LIMIT+OFFSET pagination.

MINI CODE SNIPPET

```
-- Multi-row INSERT + pagination
INSERT INTO students (name, email) VALUES
  ('Asha', 'asha@x.com'),
  ('Ravi', 'ravi@x.com');

SELECT id, name FROM students
WHERE name LIKE 'A%'
ORDER BY created DESC
LIMIT 20 OFFSET 40;
```

DAY 3 Wed Aug 26 JOINS + Aggregations

CONCEPTS

- INNER JOIN, LEFT JOIN, RIGHT JOIN, SELF JOIN
- ON clause vs USING
- GROUP BY column; COUNT, SUM, AVG, MAX, MIN
- HAVING (filter after GROUP BY); DISTINCT

READ / WATCH

Mosh MySQL Ep 4–5; Caleb Curry SQL JOINS.

HOW TO PRACTICE

Write 8 JOIN queries on school DB. Average grade per course. Count students per course. Courses with >10 students (HAVING).

MINI CODE SNIPPET

```
-- Average grade per course (LEFT JOIN + GROUP BY)
SELECT c.title, AVG(e.grade) AS avg_grade, COUNT(*) AS n
FROM courses c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id
HAVING COUNT(*) > 10
ORDER BY avg_grade DESC;
```

DAY 4 Thu Aug 27 Subqueries + Indexes + EXPLAIN

CONCEPTS

- Subquery: query inside a query — uncorrelated vs correlated
- Indexes: speed up WHERE and JOIN; downside: slower INSERT/UPDATE, more disk
- EXPLAIN shows if query uses index or does full scan
- When to add: on FK, on WHERE-often columns, on ORDER-BY columns

READ / WATCH

Mosh MySQL Ep 6–7; MySQL EXPLAIN docs.

HOW TO PRACTICE

Write 5 subquery examples. Add index on a FK. Run EXPLAIN before/after — document the difference.

MINI CODE SNIPPET

```
-- Students who scored above the course average
SELECT s.name, e.grade
FROM enrollments e
JOIN students s ON s.id = e.student_id
WHERE e.grade > (
  SELECT AVG(grade) FROM enrollments
  WHERE course_id = e.course_id
);
EXPLAIN SELECT ...; -- inspect type: ref > range > ALL
```

DAY 5 Fri Aug 28 Normalization + Views + Transactions + Stored Procs

CONCEPTS

- Normalization: 1NF (atomic), 2NF (no partial dep), 3NF (no transitive dep)
- Denormalize for read speed — tradeoff
- Foreign keys enforce referential integrity
- Views = saved SELECT, use like a table
- Transactions: BEGIN + COMMIT / ROLLBACK; ACID
- Stored procs = server-side functions in SQL

READ / WATCH

Mosh MySQL Ep 8–9; Decomplexify "Database Normalization" on YouTube.

HOW TO PRACTICE

Denormalize a table → normalize to 3NF. Create 2 views. Write bank transfer transaction (debit + credit, ROLLBACK on error). Write a paginated search stored proc.

MINI CODE SNIPPET

```
-- Money transfer transaction
START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
-- if any step fails: ROLLBACK; else:
COMMIT;
```

DAY 6 Sat Aug 29 mysql2/promise + Connection Pool

CONCEPTS

- Never use string concatenation for SQL (SQL injection!) — parameterized queries
- Connection pool: reuse N connections instead of opening on every request
- `db.js` module exporting a shared pool
- Error handling: connection errors, deadlocks, retries

READ / WATCH

mysql2 npm docs: <https://github.com/sidorares/node-mysql2>; Traversy Node MySQL on YouTube.

HOW TO PRACTICE

Set up mysql2 pool. Write `db.js` module. Run 5 parameterized queries. Handle connection errors gracefully.

MINI CODE SNIPPET

```
// db.js - shared pool
import mysql from 'mysql2/promise';
export const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  connectionLimit: 10,
});
export const query = (sql, params) =>
  pool.execute(sql, params).then(([rows]) => rows);
```

DAY 7 Sun Aug 30 Repository Pattern + Mini Project

CONCEPTS

- Repository = data access abstraction; business logic doesn't know if data lives in MySQL, Postgres, or HANA
- `findAll(filters, pagination)`, `findById`, `create`, `update`, `delete`
- Wire into Phase 7 MVC service layer
- Use weekend 6–9 PM project block to finish

READ / WATCH

Review Phase 7 MVC; blog articles on Repository Pattern in Node.

HOW TO PRACTICE

Build `StudentRepository`. Wire into Express + service layer. Full CRUD API with pagination + filtering. Push to GitHub.

MINI CODE SNIPPET

```
// StudentRepository (thin DAO)
export const StudentRepo = {
  findAll: ({ limit = 20, offset = 0 } = {}) =>
    query('SELECT * FROM students LIMIT ? OFFSET ?', [limit, offset]),
  findById: id =>
    query('SELECT * FROM students WHERE id = ?', [id]).then(r => r[0]),
  create: (data) =>
    query('INSERT INTO students SET ?', [data]),
};
```

YOUTUBE + DOCS

YT Programming with Mosh MySQL Full Course (3h, best MySQL intro): https://youtu.be/7S_tz1z_5bA

YT Caleb Curry SQL JOINS Explained on YouTube

YT Decomplexify "Database Normalization" (visual, excellent): https://youtu.be/6FQaEYE8_8

DOCS MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/> | mysql2 npm: <https://github.com/sidorares/node-mysql2>

Recommended certifications: (1) HackerRank "SQL" Basic + Intermediate + Advanced badges (30-45 min each, add all 3 to resume). (2) freeCodeCamp "Relational Database" certification. (3) Oracle MySQL 8.0 Database Developer 1Z0-909 (paid, high-value for later).

PRACTICE QUESTIONS

Q1 —

Write a query to find the top 5 students by average grade across all courses they are enrolled in.

Q2 —

Design a normalized schema (3NF) for an e-commerce app with: products, categories (many-to-many), suppliers, inventory per warehouse.

Q3 —

Write a stored procedure `search_students(p_name, p_limit, p_offset)` that returns paginated results filtered by partial name match.

Q4 —

Write the mysql2 connection pool setup with: connectionLimit:10, error event handler, a helper function `query(sql, params)` that always returns rows.

Q5 —

Explain the difference between EXPLAIN's "type" values: ALL, index, range, ref, const. Which is best and why?

MINI PROJECT — STUDENT RECORDS API

- MySQL DB: students, courses, enrollments
- Full CRUD via Express + mysql2 + Repository pattern
- Pagination, filtering, search
- Transaction for bulk enrollment
- zod validation + JWT auth from Phase 7
- Swagger docs at /api-docs

DELIVERABLES

- [] School DB schema created (Day 1)
- [] 20 rows inserted, 10 SELECT queries (Day 2)
- [] 8 JOIN + aggregation queries (Day 3)
- [] EXPLAIN before/after index report (Day 4)
- [] 3NF schema + transaction written (Day 5)
- [] mysql2 pool + db.js module (Day 6)
- [] Student Records API on GitHub (Day 7)

MINI PROJECT IDEAS — PHASE 9

1. **Student Records API** — MySQL + mysql2 pool + Repository pattern + Express + Swagger. Full CRUD, pagination, filtering, transaction for bulk enrol, JWT-protected.
2. **Library Management** — Books, members, loans schema in 3NF. Reservation with lock + transaction. Reports: overdue loans, top books, active members (JOIN + GROUP BY practice).
3. **SQL Query Playground** — A tiny web tool where you paste a SQL query, it runs against a sample school DB via Node, shows results + EXPLAIN tree side-by-side.

Break Days — Aug 31 - Sep 1 (Monday + Tuesday)

Purpose: 2-day revision of MySQL before Git + Docker + AWS.

- **Aug 31 (Mon):** Open MySQL Workbench. Without notes, design a fresh schema (e.g., library management). Write CREATE TABLE statements, add indexes, write 5 JOIN queries.
- **Sep 1 (Tue):** Review Repository pattern code. Write UserRepository from scratch. Review EXPLAIN output interpretation. Polish Student Records API README.
- Phase 10 (Git + Docker + AWS) starts fresh Wednesday Sep 2.

Git (Advanced) + Docker + AWS

3 days • Sep 2 (Wed) - Sep 4 (Fri)

What is this? The DevOps trio — version control workflows real teams use (Git), containerization (Docker), and cloud deployment (AWS). Together they turn "works on my machine" into "runs anywhere".

Why should you learn this? (1) Deploying your capstone needs Docker + AWS. (2) SAP BTP uses Cloud Foundry which is very similar to AWS ECS — the mental model transfers. (3) Docker + AWS on your resume shifts you from "coder" to "engineer".

Topics covered: Git rebase / stash / cherry-pick / bisect, PR workflow, Dockerfile, docker-compose, volumes + networks, multi-stage builds, AWS EC2 (SSH + deploy), S3 (buckets + upload), IAM (roles + policies), Lambda + CloudFront (awareness)

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Wed Sep 2 Advanced Git + Linux Basics

CONCEPTS

- Recap: clone, add, commit, push, pull, branch, merge
- rebase (rewrite history), stash (shelve WIP), cherry-pick, bisect
- PR workflow: fork → branch → PR → review → merge
- Linux basics: `ls`, `cd`, `mkdir`, `chmod`, `ssh`, `scp`, `ps`, `kill`, `tail -f`, `grep`, `sudo`

READ / WATCH

freeCodeCamp Advanced Git; Net Ninja Rebase vs Merge; Pro Git Book Ch.3: <https://git-scm.com/book/en/v2>.

HOW TO PRACTICE

Rebase a feature branch onto main. Stash mid-work changes. Cherry-pick a commit. Run `git bisect`. Write `.github/pull_request_template.md`.

MINI CODE SNIPPET

```
# Git rebase workflow
git checkout feature-x
git fetch origin
git rebase origin/main      # replay your commits on latest main
# resolve conflicts, then:
git rebase --continue
git push --force-with-lease  # safer than --force
```

DAY 2 Thu Sep 3 Docker + docker-compose

CONCEPTS

- Container = lightweight isolated process (not a VM)
- Image = frozen snapshot → Container = running instance
- Dockerfile: `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD`

- Multi-stage build: builder stage + slim runtime stage
- docker-compose: multi-container apps in one YAML
- Volumes (persist data), networks (link containers)

READ / WATCH

TechWorld with Nana Docker Full Course: <https://youtu.be/3c-iBn73dDE> ; Docker docs Get Started.

HOW TO PRACTICE

Write Dockerfile for Node Auth API (multi-stage). `docker-compose.yml` : app + MySQL + volume. `docker-compose up` and curl endpoints.

MINI CODE SNIPPET

```
# Multi-stage Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
CMD ["node", "dist/server.js"]
```

DAY 3 Fri Sep 4 AWS EC2 + S3 + IAM

CONCEPTS

- AWS = 250+ services; learn EC2, S3, IAM first
- EC2: free-tier t2.micro, SSH with .pem, install Node, run your API
- S3: bucket = folder in the cloud; upload/download via SDK
- IAM: users (humans), roles (services), policies (JSON perms); least privilege
- Lambda + CloudFront: awareness only (deep dive later)

READ / WATCH

freeCodeCamp AWS for Beginners: <https://youtu.be/uLprqHHWlng> ; AWS Getting Started tutorials.

HOW TO PRACTICE

Launch free-tier EC2, SSH in, deploy Node API. Create S3 bucket, upload multer files to S3 instead of disk. Attach IAM role.

MINI CODE SNIPPET

```
# S3 upload from Node using AWS SDK v3
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
const s3 = new S3Client({ region: 'ap-south-1' });
await s3.send(new PutObjectCommand({
  Bucket: 'my-bucket',
  Key: 'avatars/' + userId,
  Body: buffer,
  ContentType: 'image/png',
}));
```

YOUTUBE + DOCS

- YT** TechWorld with Nana Docker Full Course (MUST WATCH): <https://youtu.be/3c-iBn73dDE>
- YT** freeCodeCamp AWS Fundamentals (4h): <https://youtu.be/uLprqHHWlng>
- YT** Net Ninja Advanced Git: https://youtu.be/Uszj_k0DGsg
- DOCS** Docker docs: <https://docs.docker.com> | AWS Free Tier: <https://aws.amazon.com/free/> | Pro Git book: <https://git-scm.com/book/en/v2>

Recommended certifications: (1) **AWS Cloud Practitioner (CLF-C02)** — the ideal starter AWS cert (paid ~\$100, resume gold). (2) Docker "Docker Certified Associate" (paid). (3) freeCodeCamp "DevOps" tutorials.

PRACTICE QUESTIONS

Q1 —

What is the difference between `git merge` and `git rebase`? When should you use each in a team workflow?

Q2 —

Write a Dockerfile for a Node.js app using multi-stage build: stage 1 installs deps, stage 2 copies only production files. Result image should be under 150MB.

Q3 —

Explain the difference between an IAM Role and an IAM User. When would you use each on AWS?

DELIVERABLES

- [] Git rebase, stash, cherry-pick practiced with evidence
- [] Dockerfile + docker-compose.yml for Node API
- [] API running on EC2 (screenshot or live URL)
- [] S3 bucket file upload working from Node
- [] All 3 practice questions answered in notes

MINI PROJECT IDEAS — PHASE 10

- Dockerize Everything** — Multi-stage Dockerfile for the Auth API + `docker-compose.yml` with MySQL + volume + healthchecks. Under 150MB final image.
- Deploy to EC2** — Launch t2.micro, install Node, clone repo, run via PM2 behind Nginx reverse proxy on ports 80/443 (Let's Encrypt cert). Live URL in README.
- S3 File Service** — Modify Phase 7 avatar upload to store in S3 with proper IAM role attached to EC2 (no long-lived keys in code). Serve signed URLs.

Break Day — Sep 5 (Saturday)

Purpose: Consolidate Git + Docker + AWS. Then use this day to *ship* Capstone A.

- Push all Phase 10 code to GitHub. Run `docker-compose up` one more time end-to-end.
- Deploy the Auth API from Phase 7 to your EC2 instance behind Nginx + Let's Encrypt cert.
- Read the **Full-Stack Capstone A** brief (next page) — you'll ship it this weekend + carry into next week.
- Phase 11 (System Design) starts fresh Sunday Sep 6.

Auth-Guarded Records API — Ship It Live

Uses everything from Phase 4 → Phase 10 • Frontend + Backend + DB + API + Security + Deploy

Why this capstone before System Design? Before you learn how to *scale* a system, you should first *build one that works end-to-end*. This capstone forces you to wire every layer you've learned so far: React UI → Express API → MySQL → JWT auth → zod validation → multer uploads → Docker → EC2 → S3. Once it runs live with a URL you can share, System Design (Phase 11) will feel far less abstract.

THE PROJECT — "STUDY LOG"

IN ONE SENTENCE

A logged-in user tracks daily study sessions (topic, duration, notes, screenshot). Filter + search + export.
Deployed with HTTPS on your own domain.

REQUIRED LAYERS (MUST-COVER)

FRONTEND (REACT + TAILWIND)

- Login + Register screens; protected routes with React Router
- Sessions list (paginated) + New Session form (controlled inputs)
- Search + filter (by topic, date range); dark mode toggle
- Screenshot upload preview before submit
- Deployed on Vercel

BACKEND (NODE + EXPRESS)

- MVC: routes / controllers / services / repositories
- JWT auth (access + refresh tokens); `protect` middleware
- zod validation for every write endpoint (422 with field errors)
- multer upload → S3 (signed URLs served back)
- Global error handler; `asyncHandler` on every route

DATABASE (MYSQL)

- 3NF schema: users, sessions, topics (many-to-many)
- Foreign keys, indexes on `user_id` + `created_at`
- Repository pattern via `mysql2` pool
- Transaction: bulk import from CSV
- One stored proc: `weekly_summary(user_id)`

API DESIGN (PHASE 8)

- Versioned under `/api/v1/`
- Cursor pagination on `/sessions`
- Consistent error envelope; correct 2xx/4xx/5xx codes
- OpenAPI 3.0 spec + Swagger UI at `/api-docs`

SECURITY

- bcrypt password hashes (never plaintext)
- Helmet + CORS whitelist + rate-limited `/auth`
- Secrets in `.env` (never committed); `.env.example` in repo

DEPLOY + OPS

- Multi-stage Dockerfile; `docker-compose.yml` for local (app + MySQL)
- EC2 t2.micro; Nginx reverse proxy; Let's Encrypt cert (HTTPS)

- S3 access via IAM role attached to EC2 (no long-lived keys in code)
- Input validation at every boundary (zod)

- PM2 process manager; log rotation
- GitHub Actions: run tests + build image on push to main
- README with architecture diagram, live URL, demo GIF

MILESTONES (WEEKEND + WEEKDAYS AFTER PHASE 11)

1. **Sep 5-6 (weekend after Phase 10):** Schema + backend skeleton + JWT auth + first two endpoints deployed to EC2. HTTPS working.
2. **During Phase 11 evenings (Sep 6 - Sep 8):** Wire frontend to backend. Login → sessions list → new session works end-to-end.
3. **Sep 9 (Break Day):** Screenshot upload → S3 signed URL. Filtering + pagination. Fix rough edges.
4. **Final weekend (Sep 13-14) if needed:** Polish README, record 2-min demo video, share the live URL.

STRETCH GOALS

1. **CSV import/export** — bulk import 100 sessions via transaction; export filtered results as CSV.
2. **Email digest** — nightly cron / n8n workflow that emails yesterday's summary via Nodemailer.
3. **Public share page** — read-only `/u/:username` page (no auth) showing top 5 topics of the last month.

System Design Fundamentals

3 days • Sep 6 (Sun) - Sep 8 (Tue)

What is this? System design is about how to structure large software systems so they scale to millions of users without falling over. Not about writing code — about drawing boxes, arrows, and choosing tradeoffs.

Why should you learn this? (1) Every mid-level+ interview asks a system design question. (2) When you enter SAP BTP, understanding load balancers, caching, and queues helps you use SAP services correctly. (3) It teaches you to think in tradeoffs, not absolutes — a senior-engineer mindset.

Topics covered: Vertical vs horizontal scaling, load balancers, monolith vs microservices, caching (Redis), CDN, message queues (Kafka/RabbitMQ concepts), CAP theorem, read replicas, sharding, API gateway

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Sun Sep 6 Scaling + Load Balancing + Architectures

CONCEPTS

- Vertical scaling (bigger server, limited) vs horizontal scaling (more servers, cheaper)
- Load balancer distributes traffic (Nginx, HAProxy, AWS ALB)
- Monolith vs Microservices — tradeoffs
- Stateless vs stateful services

READ / WATCH

ByteByteGo channel: <https://www.youtube.com/@ByteByteGo> ; Fireship "Microservices vs Monolith".

HOW TO PRACTICE

Draw diagram: monolith Student Records API → split into microservices. Add load balancer in front. Write pros/cons for each.

MINI CODE SNIPPET

```
# Draw this in your notes
[Users]
|
[LB Nginx]
|-----|-----|
[API1][API2][API3]    ← horizontal scale
|
[MySQL primary]
+→ [MySQL replica] ← reads
```

DAY 2 Mon Sep 7 Caching + CDN + Message Queues

CONCEPTS

- Cache = fast storage in front of slow storage (Redis in front of MySQL)
- Cache patterns: cache-aside, write-through, write-behind
- TTL, invalidation, "stampede" problem

- CDN = geographically distributed static file cache (CloudFront, Cloudflare)
- Message queue = async pipe (Kafka, RabbitMQ, SQS); Producer/Consumer decoupling

READ / WATCH

ByteByteGo Redis + CDN videos; "Message Queues in 5 Min" on YouTube.

HOW TO PRACTICE

Design caching strategy for Student Records API: what to cache, TTL, invalidation. Draw CDN + queue architecture for async notifications.

MINI CODE SNIPPET

```
# Cache-aside pattern (pseudo)
function get(id):
    v = cache.get(id)
    if v: return v          # HIT
    v = db.query('...', id) # MISS
    cache.set(id, v, ttl=60)
    return v
```

DAY 3 Tue Sep 8 CAP Theorem + Sharding + API Gateway

CONCEPTS

- CAP: pick 2 of Consistency / Availability / Partition-tolerance
- Read replicas: clones of DB for read scaling (writes go to primary)
- Sharding: split data across DBs by key range or hash
- API Gateway: single entry-point (auth, rate limit, routing)

READ / WATCH

ByteByteGo CAP Theorem; "Database Sharding" article; Gaurav Sen Consistent Hashing on YouTube.

HOW TO PRACTICE

Write 1-page system design doc: "Scale Student Records API to 1M users". Include load balancer, read replicas, cache, CDN, async jobs. Save to GitHub.

MINI CODE SNIPPET

```
# Shard by user_id hash
shard_id = hash(user_id) % 4
db_url   = SHARD_URLS[shard_id]

# API Gateway responsibilities
- Auth (JWT verify)
- Rate limit (100 req/min)
- Route to internal services
- Response caching (short TTL)
```

YOUTUBE + DOCS

YT ByteByteGo channel (Alex Xu — every video is gold): <https://www.youtube.com/@ByteByteGo>

YT Gaurav Sen channel: <https://www.youtube.com/@gkcs>

YT freeCodeCamp System Design Interview (2h): <https://youtu.be/i7twT3x5yEI>

DOCS Alex Xu "System Design Interview" book + newsletter: <https://blog.bytebytego.com>

Recommended certifications: No formal beginner cert. Instead: (1) Read "System Design Interview Vol 1" by Alex Xu (book). (2) Attempt 3 free case studies on the ByteByteGo newsletter and publish your write-up on GitHub — that IS the portfolio artifact.

PRACTICE QUESTIONS

Q1 —

Design a URL shortener (like bit.ly). Cover: DB schema, ID generation, read/write split, caching layer, CDN.

Q2 —

Explain CAP theorem. In a system that requires high availability (e.g., Twitter feed), which two properties do you choose and what do you sacrifice?

Q3 —

When would you use horizontal sharding vs read replicas? Give a concrete example where you need both.

DELIVERABLES

- ☐ Monolith → microservices diagram drawn
- ☐ Caching strategy document written
- ☐ "Scale to 1M users" system design doc saved to GitHub

MINI PROJECT IDEAS — PHASE 11

- 1M-User Design Doc** — One-page design: scale Study Log API to 1M users — LB + read replicas + Redis cache + CDN + async jobs + monitoring.
- URL Shortener at Scale** — Design doc for a bit.ly clone: ID generation (base62 vs snowflake), write path, read cache, analytics pipeline, sharding at 100M links.
- SAP-flavoured Design** — Design a Fiori launchpad backend that scales for a large enterprise (SSO, tile caching, personalization). Ties Phase 11 to Part II/III.

Break Day — Sep 9 (Wednesday)

Purpose: Wrap Capstone A + prep the mind for the final AI + Capstone phase.

- Finish Capstone A polish + push to GitHub with a great README
- Read the **Full-Stack Capstone B** brief (next page) — that's your target for the final weekends (Sep 13-14)
- Read the Claude API "tool_use" documentation (30 min familiarization before Phase 12)
- Set up n8n locally: `npx n8n` — verify the UI opens at localhost:5678
- Phase 12 (final!) starts fresh Thursday Sep 10.

Study Log — Scaled, Cached & Observed

Takes Capstone A + applies every System Design lesson from Phase 11 • Target: final Part I weekends (Aug 30 / Sep 6-7 / Sep 13-14)

Why this capstone after System Design? Phase 11 taught you scaling *concepts*. This capstone forces you to actually *apply* them to your own live app. You'll add caching, background jobs, monitoring, and CI/CD to the Auth-Guarded Records API you shipped in Capstone A. When an interviewer asks "how do you handle 100k users?", you'll have a real repo to point at.

THE UPGRADE — FROM CAPSTONE A TO CAPSTONE B

CACHING LAYER

- Add Redis (Docker Compose service)
- Cache-aside on `GET /sessions/:id` (TTL 5 min)
- Invalidate on PUT/DELETE
- Response header `X-Cache: HIT|MISS`

ASYNC / QUEUE WORK

- Background job: nightly summary via BullMQ + Redis
- Retry with exponential backoff on failure
- Dead-letter queue for permanent failures

DATABASE SCALING

- Add read-replica config (or fake with a 2nd MySQL container)
- Route SELECTs to replica, writes to primary via a wrapper
- Add composite indexes based on your slowest EXPLAIN plans

OBSERVABILITY

- Structured logging with pino (JSON logs)
- Prometheus `/metrics` endpoint (request count, latency histogram)
- Grafana dashboard: p50 / p95 / error rate / cache hit ratio
- Alert rule: p95 > 1s for 5 minutes → email

CDN + FRONTEND PERF

- CloudFront in front of S3 for static uploads
- Code splitting + lazy routes in the React app
- Lighthouse target: performance ≥ 90

CI/CD HARDENING

- GitHub Actions: test → build image → push to ECR
- Deploy step SSHes to EC2, runs `docker compose pull && up -d`
- Rollback command in README
- Signed commit protection on main branch

INTERVIEW STORY YOU CAN TELL AFTER THIS

1. "I moved `/sessions` reads from 400ms to 15ms by adding a Redis cache-aside layer with 5-minute TTL — measured with a Grafana p95 chart."
2. "When SELECTs started dominating my primary DB, I split reads to a replica; that dropped the primary CPU by ~40%."
3. "Nightly summaries move to a BullMQ queue so a slow email provider never blocks a user request."

Agentic AI + n8n + Capstone Project

5 days • Sep 10 (Thu) - Sep 14 (Mon) • FOUNDATION COMPLETE

What is this? The finale: build a full-stack AI-powered task manager (FocusTrack Pro) using Claude API with tool use, RAG, and an n8n automation workflow. Everything from the previous 11 phases converges here.

Why should you learn this? This project is your *portfolio centrepiece* for job applications. It shows: React + Node + MySQL + Docker + AWS + Claude API + tool use + RAG + workflow automation — the exact stack a 2027 "SAP + AI Engineer" role wants.

Topics covered: Claude API basics, `messages.create`, tool use / function calling, agent loop with `maxIterations` guard, RAG (SQL-backed retrieval), n8n workflows (schedule trigger, webhook, AI nodes, MySQL node), full-stack integration, Docker deploy, portfolio-quality README

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY 1 Thu Sep 10 Claude API: `messages.create` + Tool Use

READ / WATCH

Anthropic docs "Tool Use" (full guide); "Build AI Agents" on Anthropic YouTube.

HOW TO PRACTICE

Make your first Claude API call from Node.js. Implement a tool `get_weather(city)` (mock return). Build a simple agent that decides when to call the tool.

MINI CODE SNIPPET

```
import Anthropic from '@anthropic-ai/sdk';
const client = new Anthropic();
const tools = [{
  name: 'get_weather',
  description: 'Return current weather for a city',
  input_schema: { type: 'object', properties: { city: { type: 'string' } }, required: ['city'] }
}];
const msg = await client.messages.create({
  model: 'claude-sonnet-4-6', max_tokens: 512,
  tools, messages: [{ role: 'user', content: 'Weather in Bangalore?' }],
});
```

DAY 2 Fri Sep 11 Agent Loops + `maxIterations` Guard + RAG (SQL-based Retrieval)

READ / WATCH

Anthropic docs "Building Agents"; "RAG Explained" IBM Technology YouTube.

HOW TO PRACTICE

Build an agent loop: query Claude → if `tool_use` in response, call tool → feed result back → repeat until text response or `maxIterations`. Build SQL-RAG: store task notes in MySQL, retrieve similar with LIKE/FULLTEXT, inject into prompt.

MINI CODE SNIPPET

```
async function runAgent(userMsg, tools, max = 5) {
  const messages = [{ role: 'user', content: userMsg }];
  for (let i = 0; i < max; i++) {
    const r = await client.messages.create({ model, tools, messages, max_tokens: 1024 });
    messages.push({ role: 'assistant', content: r.content });
    const use = r.content.find(c => c.type === 'tool_use');
    if (!use) return r; // text reply - done
    const out = await runTool(use.name, use.input);
    messages.push({ role: 'user', content: [{ type: 'tool_result', tool_use_id: use.id, content:
  }
}
```

DAY 3 Sat Sep 12 n8n Workflows: Schedule + Webhook + AI Nodes + MySQL

READ / WATCH

n8n official YouTube (getting started series); n8n docs "AI nodes".

HOW TO PRACTICE

Build n8n workflows: (1) Daily 8am schedule → query MySQL for today's tasks → generate summary with Claude AI node → send email/webhook. (2) Webhook trigger from app → AI classifies task priority → update MySQL.

MINI CODE SNIPPET

```
# n8n daily summary workflow (nodes)
[Schedule 8:00 AM]
↓
[MySQL: SELECT tasks WHERE user_id=? AND due=TODAY]
↓
[Claude AI: summarize + prioritize]
↓
[Send Email / Webhook]
```

DAY 4 Sun Sep 13 Capstone: FocusTrack Pro — Backend + Frontend Integration

READ / WATCH

Your own Phase 6 React code + Phase 7 Express API + Phase 9 MySQL + Claude API docs.

HOW TO PRACTICE

Wire up: React frontend (Task Tracker from Phase 6) → Express backend (Auth API from Phase 7) → MySQL (Student Records DB adapted) + Claude AI endpoint. Add an "AI Suggest" button that calls Claude to suggest next task based on context.

MINI CODE SNIPPET


```
// Express route calling Claude with SQL-RAG
app.post('/ai/suggest', asyncHandler(async (req, res) => {
  const ctx = await query(
    'SELECT title, status FROM tasks WHERE user_id=? ORDER BY created DESC LIMIT 5',
    [req.user.id]);
  const r = await client.messages.create({
    model, max_tokens: 300,
    messages: [{ role: 'user', content: `Given ${JSON.stringify(ctx)}, suggest the next best task` }]);
  res.json({ suggestion: r.content[0].text });
}));
```

DAY 5 Mon Sep 14 Final Polish, Docker Deploy, README, Portfolio Prep

READ / WATCH

GitHub portfolio best practices; "How to Write a Good README" article.

HOW TO PRACTICE

Deploy FocusTrack Pro: `docker-compose up` on EC2, frontend on Vercel, API on EC2. Write a great README (screenshots, tech stack, setup, architecture diagram). CELEBRATE — 50-day foundation COMPLETE!

MINI CODE SNIPPET

```
# docker-compose.yml - full stack
services:
  api:
    build: .
    ports: ["3000:3000"]
    depends_on: [db]
    env_file: .env
  db:
    image: mysql:8
    environment: { MYSQL_ROOT_PASSWORD: secret, MYSQL_DATABASE: focustrack }
    volumes: ["db-data:/var/lib/mysql"]
volumes:
  db-data:
```

YOUTUBE + DOCS

YT Anthropic YouTube (Build AI Agents with Claude): <https://www.youtube.com/@AnthropicAI>

YT n8n official YouTube (Getting Started): <https://www.youtube.com/@n8n-io>

YT RAG from Scratch (freeCodeCamp / LangChain playlists on YouTube)

DOCS Claude Tool Use docs: <https://docs.anthropic.com/en/docs/build-with-claude/tool-use>

DOCS n8n docs: <https://docs.n8n.io> | n8n AI nodes: <https://docs.n8n.io/advanced-ai/>

Recommended certifications: (1) DeepLearning.AI "AI Agents in LangGraph" (free audit) — conceptually same as Anthropic agents. (2) Anthropic "Claude API Fundamentals" — if launched by then, take it. (3) The *real* credential here is your GitHub: pinned FocusTrack Pro with a stellar README = beats most certs.

PRACTICE QUESTIONS

Q1 —

Write a Claude API call with `tool_use` for a `create_task(title, priority, dueDate)` function. Show the full request and how you parse the `tool_use` response.

Q2 —

Implement an agent loop function: `runAgent(userMessage, tools, maxIterations=5)`. Handle the STOP condition when Claude returns a text response instead of `tool_use`.

Q3 —

Design an n8n workflow that: receives a webhook when a task is marked done → checks if user has 0 remaining tasks for today → sends a congratulations message via email. Draw the node graph.

Q4 —

Build a SQL-RAG function: `getRelevantContext(userQuery)` that does a FULLTEXT search on a `task_notes` table and returns the top 3 matching notes as a string to inject into a Claude prompt.

Q5 —

What is the difference between a ReAct agent and a simple tool-use agent? How would you implement a basic ReAct loop using the Claude API?

CAPSTONE: FOCUS TRACK PRO

- **Frontend:** React + Tailwind (from Phase 6)
- **Backend:** Node.js + Express + MySQL (Phases 7-9)
- **AI:** Claude API with tool use (`create_task`, `get_suggestions`)
- **Automation:** n8n daily summary + priority classification
- **Deploy:** EC2 + Docker + Vercel frontend
- **Auth:** JWT from Phase 7
- **Docs:** Swagger at `/api-docs`

FOUNDATION COMPLETE — FINAL CHECKLIST

- ☐ FocusTrack Pro deployed and accessible
- ☐ GitHub repo with 12 phase folders
- ☐ Prompt Playbook (Phase 1)
- ☐ CLAUDE.md set up for repo
- ☐ Task Tracker React app (Phase 6)
- ☐ Auth API with Swagger docs (Phase 7-8)
- ☐ Student Records MySQL API (Phase 9)
- ☐ System Design doc (Phase 10)
- ☐ Docker + EC2 deployment (Phase 11)
- ☐ FocusTrack Pro full-stack + AI (Phase 12)
- ☐ GitHub profile polished with pinned repos
- ☐ Ready for Part II: SAP Overview (Sep 15-17) + ABAP institute (orientation Sep 18-20)

Foundation Complete!

49 study days • Sep 14, 2026

Next: Part II — SAP Overview (Sep 15-17), institute orientation Sep 18-20. Ship Capstones C & D on the final weekends.

MINI PROJECT IDEAS — PHASE 12

1. **FocusTrack Pro (Capstone)** — React + Node + MySQL + Claude API tool use + n8n automation, deployed via Docker to EC2 + Vercel. AI Suggest + Daily Summary features. THIS is your portfolio centrepiece.
2. **Personal Research Agent** — Node CLI: takes a topic, uses tools (web fetch, note write) in an agent loop with maxIterations=5, outputs a Markdown research brief. *(Full brief on next page — Capstone C.)*
3. **SQL-RAG Q&A** — Small app: your task notes stored in MySQL; user asks a question; FULLTEXT retrieval top-3 → injected into Claude prompt → grounded answer with citations. *(Extended brief — Capstone D.)*

Personal Research Agent — Multi-Tool CLI

Uses Claude API + tool use + agent loop • Target: final Part I weekends (Sep 6-7 / Sep 13-14)

Why this project? Phase 12 taught you the anatomy of an agent loop. Capstone C forces you to build a real one that *does useful work* from your terminal — picking tools, calling them, and stopping cleanly. It's the smallest possible interview showcase for "yes, I've built AI agents."

THE PROJECT — "RESX"

IN ONE SENTENCE

A Node CLI: `resx "Explain SAP AI Core vs Azure OpenAI"`. The agent picks from tools (web fetch, notes read/write, code sandbox), loops with `maxIterations=5`, and writes a Markdown research brief with citations.

REQUIRED CAPABILITIES

TOOLS (3 MINIMUM)

- `web_fetch(url)` — download a URL, return trimmed text
- `notes_write(path, content)` — persist to `./notes/`
- `notes_search(query)` — grep across existing notes

AGENT LOOP

- `maxIterations=5`; halt on text-only response
- Log every tool call + result to a run log
- Cost + token counter printed at the end
- Graceful error handling on tool failures

OUTPUT

- Markdown brief with H2 sections + inline citations `[1]`, `[2]`
- Sources list at bottom (URLs actually fetched)
- Saved under `./briefs/{timestamp}-{slug}.md`

CLI ERGONOMICS

- `resx "topic"` = quick brief
- `resx --deep "topic"` = 10 iterations, more sources
- `resx list` = list past briefs
- Pretty progress bar while iterations run

STRETCH GOALS

1. **Prompt caching** — enable ephemeral cache on the system prompt / tool defs; measure token savings.
2. **Streaming output** — use `stream: true` so partial responses render live in the terminal.
3. **Local vector search** — replace `notes_search` grep with a real embedding index (SQLite + `sqlite-vec`).

RAG Assistant + n8n Automations — Full Loop

Uses Claude + vector search + n8n workflows • Target: final Part I weekend (Sep 13-14)

Why this project? Capstone C proved you can build an agent. Capstone D proves you can plug one into a *system* — async pipelines, retrieval, human-in-the-loop workflows. This is the exact pattern SAP Joule custom skills use, so it doubles as a stepping-stone to Part III.

THE PROJECT — "ASKMYNOTES"

IN ONE SENTENCE

A web app + n8n automation stack: you dump study notes into a folder; a workflow embeds and stores them; a chat UI lets you ask questions and get grounded answers with citations. Daily digests come via email.

REQUIRED LAYERS

INGESTION PIPELINE (N8N)

- Trigger: file drop into `./inbox/` or a Google Drive folder
- Chunk text (500 tokens with 50 overlap)
- Embed each chunk (Voyage / OpenAI / Anthropic embedding)
- Store in MySQL FULLTEXT + a vector table (sqlite-vec or pgvector)

RAG API (EXPRESS)

- `POST /ask` — embed question → vector search top-5 → inject as context → Claude
- Cite chunks by source file + line range
- Rate-limited + JWT-protected
- Log every Q/A pair for later evaluation

CHAT UI (REACT)

- Chat pane with streaming responses
- Citations rendered as clickable footnotes
- "Show me source" opens the referenced chunk
- Feedback thumbs up/down per answer

DAILY AUTOMATION (N8N)

- 7 AM cron: query MySQL for today's roadmap topic
- Ask the RAG for a 5-bullet summary of what you last studied
- Email it to yourself (Nodemailer) with a "study now" link
- Log every run for the "did I open the mail?" tracker

STRETCH GOALS

1. **Multi-tenant** — make it work for you + a friend, each with their own vector namespace and API key.
2. **SAP-flavour port** — in Part III, port the vector table to HANA Vector Engine and wrap the ask endpoint as a Joule custom skill. Instant SAP portfolio piece.
3. **Evaluation harness** — a set of 20 golden Q/A pairs; nightly cron re-runs them and reports drift.

You now have 4 portfolio-grade capstones.

A (auth-guarded app), B (scaled + observed), C (agent CLI), D (RAG + automations). Any one of these beats a certificate.

SAP Overview + ABAP on HANA (Institute)

Sep 15, 2026 → institute wrap (end Nov / mid Dec) • 3-day SAP crash-course + institute-paced ABAP course

Part II has two halves. **Sep 15–17** is a fast, self-paced SAP *orientation* — three no-code days to get the vocabulary, module landscape, and career map into your head before joining an institute, and to **finalise enrolment**. Shortlisting + fee comparison should already be done during the JS reset week (Jul 20-26) and the final Part I weekends, so these three days are pure orientation plus signing. **Sep 18–20** is institute orientation, and the ABAP course proper runs from there (Future Labs or equivalent, 40 modules, 60 hours). SAPUI5 basics are covered inside the institute curriculum so no separate UI5 phase is needed.

No standalone buffer week. The old Sep buffer week is gone — that slack moved earlier into the **JS reset week (Jul 20–26)**. Phase 12 ends **Mon Sep 14**, and SAP starts the very next day. Wrap and deploy your Part I capstones (B, C, D) on the *final weekends* of Phase 12, not after — there's no catch-up week on this side anymore.

MASTER TIMELINE

BLOCK	DATES	WHAT HAPPENS
Phase A	Sep 15-17 (3 days, Tue-Thu)	SAP Overview & Ecosystem — self-paced crash-course. No coding. Get oriented: SAP + ERP basics, module landscape, S/4HANA vs ECC, HANA, BTP, Fiori/UI5, career paths + certifications. Also: finalise institute enrolment.
Orientation	Sep 18-20 (3 days, Fri-Sun)	Institute Kickoff — joining formalities, SAP system credentials, Eclipse ADT / SAP GUI setup, first Hello World program.
Phase B	Sep 21 → end Nov / mid Dec (institute-paced)	ABAP Institute Course — 40 modules, 60 hours. <i>Do not treat the module segments below as fixed dates — institute pace decides.</i> Vendor Hub side project runs alongside.

Important: Institute Pace > Roadmap Pace

You have **not joined the institute yet**. Their batch schedule, homework load, and topic sequence will determine your actual pace — not this document. So the week-by-week module map below is a **reference outline**, not a daily commitment. Two things follow from that:

- Do NOT plan day-by-day activities in your organiser app for institute weeks — wait until you have the batch schedule.
- Course completion window: **end Nov to mid Dec** depending on institute intensity.
- Part III (SAP BTP + AI self-pace) starts **Dec 1 regardless** — some overlap with the tail of ABAP is fine and expected.

ABAP COURSE MODULE OUTLINE (INSTITUTE-PACED, REFERENCE ONLY)

SEGMENT	APPROX TIMING	MODULES	FOCUS
Orientation	First 2-3 days after joining	Institute joining	Enrolment, system access, IDE (Eclipse ADT / SE38) setup, first Hello World

SEGMENT	APPROX TIMING	MODULES	FOCUS
Segment 1	~Week 1	M1-M4	ABAP Workbench, SE38/ADT, Data Types, Control Flow, Field Strings
Segment 2	~Week 2	M5-M8	ABAP Dictionary: Tables, Structures, Views, Data Elements, Domains, Packages
Segment 3	~Week 3	M9-M12	Search Helps, Lock Objects, Open SQL SELECT / JOINS / Subqueries
Segment 4	~Week 4	M13-M16	Internal Tables (Standard/Sorted/Hashed), Debugging, Selection Screens
Segment 5	~Week 5	M17-M20	Modularization: Subroutines, Function Modules, Function Groups, Includes, Packages
Segment 6	~Week 6	M21-M24	Reports: Classical, Interactive, ALV (CL_SALV_TABLE)
Segment 7	~Week 7	M25-M28	Dialog / Module Pool: Screen Painter, PBO/PAI, POV/POH, Tab Strips
Segment 8	~Week 8	M29-M31	OO ABAP: Classes, Inheritance, Polymorphism, Interfaces, Events
Segment 9	~Week 9	M32-M34	BDC (Recording, Call Transaction, Session Method), LSMW
Segment 10	~Week 10	M35-M37	SAP Scripts, Smart Forms, RFC, ALE, IDocs, EDI
Segment 11	~Week 11	M38-M39 + UI5 intro	BAPIs, User Exits, BADIs, Enhancement Framework, CTS + SAPUI5 basics (institute-taught)
Wrap	Final week	M40	ABAP on HANA — CDS Views, AMDP, ADBC, Code-to-Data. Vendor Hub demo.

Side Project: Vendor Hub (runs all 12 weeks)

A Vendor Management System in ABAP. Each week adds a module: Wk 1 basic report → Wk 2 custom table → Wk 4 selection screen → Wk 6 ALV → Wk 7 dialog app → Wk 8 OO refactor → Wk 9 BDC upload → Wk 11 Fiori/UI5 wrapper → Wrap: HANA CDS view. By Nov 30 you have a complete, demo-able SAP application.

DAILY SCHEDULE (WEEKDAYS) — INSTITUTE PHASE

- 8:45 – 11:20 AM — SAP theory + brochure reading + practice on SAP system
- 12:00 – 9:00 PM — Office (Mediacle)
- 10:00 – 11:15 PM — Vendor Hub side project OR institute homework
- **Note:** Institute may schedule Sat/Sun classes — that replaces your usual weekend study block on those days.

SAP SYSTEM ACCESS

- SAP ABAP Developer Edition (free download from SAP)
- OR: SAP BTP Free Trial — ABAP Cloud Environment
- IDE: Eclipse ADT (preferred) or SAP GUI (SE38/SE80)
- Institute usually provides system credentials for a live SAP server

SAP Overview & Ecosystem Orientation

Sep 15-17, 2026 (Tue-Thu) • Self-study • No coding • Finalise institute enrolment

What is this? A fast, no-code crash-course on the SAP world — company, product landscape, ERP concepts, modules, technology stack, and where an ABAP + BTP + AI engineer fits into it.

Why should you learn this? When you walk into the institute, the trainer will assume you already know what "S/4HANA", "Fiori", "FI/CO", and "GUI vs Web IDE" mean. Learning the vocabulary *before* the course lets you spend your attention on ABAP itself instead of on context.

Why only 3 days? This is *awareness-level* reading, not skill-building — no code, no exercises. The institute teaches the depth. Phase 12 ends Sep 14 and there's no buffer week anymore, so this is tight by design: shortlisting + fee comparison should already be done (JS reset week + final Part I weekends), leaving these three days for orientation plus signing the enrolment form.

Topics covered: SAP history; ERP fundamentals; SAP module landscape (FI, CO, MM, SD, PP, HR); S/4HANA vs ECC; SAP HANA (in-memory DB); Cloud vs on-prem + BTP; Fiori + UI5 tour; ABAP career paths; SAP certifications map

DAY-BY-DAY — STARTING FROM PURE BASICS

DAY 1 Tue Sep 15 SAP + ERP + Modules + S/4HANA vs ECC + HANA

CONCEPTS — COMPANY & ERP

- SAP = **S**ystems, **A**pplications & **P**roducts; German, founded 1972 by five ex-IBM engineers; #1 ERP vendor (~77% of world transaction revenue touches SAP)
- ERP = *one* system for finance + HR + supply chain + sales; modules talk to each other (an SD sale auto-creates the FI journal entry)
- Family beyond S/4HANA: SuccessFactors (HR), Ariba (procurement), Concur (travel), Fieldglass

CONCEPTS — MODULES, S/4HANA, HANA

- **Functional:** FI, CO, MM, SD, PP, HR/HCM, PM, QM • **Technical:** ABAP, BASIS, BW/BI, Security, PI/PO/CPI, Fiori/UI5
- **R/3 / ECC** = old client-server era (1992, ECC 6.0); **S/4HANA** = 2015 rewrite, HANA-only, Fiori-first, simplified data model
- **2027 deadline:** SAP ends mainstream ECC support → huge migration wave → exactly why the job market is hot for you
- **HANA:** stores *columns in RAM* (vs rows on disk) — huge analytics speed; "code-to-data" pushes logic into the DB (you'll do this in the wrap segment)

READ / WATCH

"What is SAP?" — SimpliLearn: <https://youtu.be/YKD1oEcNvpM> ; "ERP in 100 Seconds" (Fireship); "SAP Modules Overview" (ZaranTech/SimpliLearn); "S/4HANA vs ECC" on SAP's official YouTube; SAP HANA Academy.

WHAT TO DO

1-page SAP history + products note. Draw the 5 core processes (O2C, P2P, H2R, P2P-produce, R2R) and map modules to each. Build a 2-column module table. Write 5 bullets on why the 2027 deadline matters + a short HANA column-store note. (This is a heavier first day — it merges what used to be two.)

DAY 2 Wed Sep 16 Cloud vs On-Prem, SAP BTP, Fiori & SAPUI5

CONCEPTS

- On-prem = customer's own data centre; Cloud = SAP hosts it; Hybrid = a mix (most real customers)
- **BTP** = Business Technology Platform — the "cloud PaaS" side of SAP
- BTP hosts your custom apps, integrations, AI models, and extensions (this is all of Part III)
- What runs where: S/4 core + ABAP on-prem; Fiori launchpad, Joule AI, CAP apps on BTP
- **Old UI**: SAP GUI — the grey, keyboard-driven Windows client
- **New UI**: Fiori = a design language + a set of pre-built apps
- **SAPUI5** = the JavaScript framework Fiori is built on (think React, but SAP-specific — your Phase 6 React knowledge transfers directly)
- Fiori launchpad = the home page of tiles a business user actually lands on

READ / WATCH

SAP BTP overview: <https://www.sap.com/products/technology-platform.html> ; "What is SAP BTP" (YouTube); SAP Fiori design demo: <https://experience.sap.com/fiori-design/> .

WHAT TO DO

Note down what runs on-prem vs on BTP — this preps you for Part III. Then open the Fiori design demo site and click through a few apps; screenshot the visual patterns you notice. You'll build UI5 apps yourself in institute Segment 11.

DAY 3 Thu Sep 17 Career Paths, Certifications & Enrolment Day

CONCEPTS

- **Career tracks**: traditional ABAP dev (reports, dialog, enhancements) → ABAP + Fiori/UI5 (full-stack SAP) → ABAP on HANA (CDS, AMDP) → BTP developer (CAP + Fiori + AI)
- Also: functional consultant + ABAP crossover; freelance vs India Big-4 vs SAP Labs vs product companies
- **Certs to know**: C_TAW12_750 (ABAP with NetWeaver), C_HANDEV_18 (ABAP on HANA 2.0), Fiori Application Developer, BTP Extension Developer
- Certs cost roughly ₹40-50K each in India — plan for 1-2 in 2027, after you land the job
- openSAP courses are **free** and give you badges you can put on a resume today

READ / WATCH

Search LinkedIn for "SAP ABAP Developer" job descriptions — read 5-10 postings to spot the common skill combos and salary bands. SAP certification catalog: <https://training.sap.com/certification> .

WHAT TO DO

Enrol. Sign with your chosen institute, pay the fee, collect system-access credentials. Install SAP GUI + Eclipse ADT tonight. Bookmark 2 certifications for 2027 and sign up for the free openSAP "ABAP Development for SAP HANA" course. Write a 1-page 5-year SAP career map. You are ready for orientation on **Fri Sep 18**.

YOUTUBE + DOCS (BOOKMARK THESE FOR THE WHOLE COURSE)

- YT** SAP official: <https://www.youtube.com/@SAP>
- YT** SAP Learning: <https://www.youtube.com/@SAPLearning>
- YT** SAP HANA Academy: <https://www.youtube.com/@saphanaacademy>
- DOCS** SAP Help Portal (bookmark): <https://help.sap.com>
- DOCS** SAP Developers hub (tutorials): <https://developers.sap.com>

Recommended certifications (free, do NOW): (1) openSAP: "ABAP Development for SAP HANA" (free, self-paced, course badge). (2) openSAP: "Fiori for SAP S/4HANA" (free). Both count as SAP-verified learning on your resume.

DELIVERABLES CHECKLIST

- [] 1-page SAP overview notes (Day 1)
- [] 5 core business processes diagram (Day 1)
- [] SAP modules table — name → what it does (Day 1)
- [] S/4HANA vs ECC + 2027 deadline note (Day 1)
- [] HANA column-store 1-pager (Day 1)
- [] On-prem vs BTP notes (Day 2)
- [] Fiori/UI5 tour screenshots (Day 2)
- [] 5-year SAP career map (Day 3)
- [] 2 certifications bookmarked + openSAP signup (Day 3)
- [] **Institute enrolment DONE + SAP GUI / Eclipse ADT installed (Day 3)**

INSTITUTE SHORTLIST CRITERIA

Do this during the JS reset week (Jul 20-26) + final Part I weekends so Day 3 is just signing.

- Curriculum covers M1-M40 + HANA CDS + AMDP
- Trainer has 5+ years SAP experience
- Live SAP system access provided
- Batch timing fits your 8:45-11:20 AM slot OR evening/weekend
- Placement support (interview practice, resume review)
- Recorded sessions available for revision

ABAP Course Kickoff

Sep 18-20, 2026 (Fri-Sun) • Enrolment complete, system access ready • Course proper starts ~Sep 21

Institute joining formalities, SAP system credentials, IDE setup, first Hello World. Follow the institute's onboarding — not a rigid script from this doc. If your batch actually begins on a different date, shift these 3 days with it. Once the course starts, use the segments below only as a *reference outline* for what should be covered by the time you finish each block.

- **Fri Sep 18 — joining day:** Collect materials, system credentials, batch schedule. Install SAP GUI + Eclipse ADT.
- **Sat Sep 19:** Log into the SAP system, run SE80 Object Navigator, create a Z-namespaces, write your first "Hello World" ABAP program with a WRITE statement.
- **Sun Sep 20:** Explore SE38 / SE11 / SE80 / SE84. Get familiar with the keyboard shortcuts. Set up your local Vendor Hub notes folder + GitHub-style ABAP repo.
- **From ~Mon Sep 21:** Course proper begins — Segment 1 (M1-M4). Follow your batch's pace, not this document's.

ABAP Workbench & Language Basics

Institute-paced • Brochure Modules M1–M4

Get comfortable in the SAP ABAP development environment. Learn core language syntax: data types, control flow, field strings. Write your first programs.

BROCHURE MODULES COVERED

M1: Intro to SAP & ABAP

M2: ABAP Workbench (SE38, SE80)

M3: Data Types & Control Flow

M4: Field Strings (Structures)

Topics to cover by end of segment

- SAP 3-tier architecture, R/3 layers, ABAP overview
- SE38/ADT setup, first program, `WRITE` statement, SE80 Object Navigator
- Elementary data types: I, F, P, C, N, D, T, X, STRING — PACKED and FLOAT precision
- Control flow: `IF/ELSEIF/ELSE`, `CASE`, `DO`, `WHILE`, `LOOP`
- Field Strings (Structures): `TYPES`, `DATA`, `BEGIN OF ... END OF`, nested structures
- SY system variables (`SY-SUBRC`, `SY-UNAME`, `SY-DATUM`), message classes, variants

Practice targets (do at institute pace, whenever these topics come up)

- Write "Hello World" ABAP program; explore SE80 Object Navigator
- Declare all data types, assign values, `WRITE` each
- FizzBuzz in ABAP; factorial using `DO`; prime checker using `WHILE`
- Define `VENDOR` structure (id, name, city, category); `WRITE` all fields
- Use `SY-SUBRC`, `SY-UNAME`, `SY-DATUM` in a small report; create a variant

YOUTUBE + DOCS

YT SAP Training & Adoption "ABAP Tutorial for Beginners" on <https://www.youtube.com/@SAPLearning>

YT ZaranTech ABAP series (search YouTube for recent 2024/2025)

DOCS Official ABAP Language docs: https://help.sap.com/doc/abapdocu_latest_index.htm

PRACTICE QUESTIONS

Q1 —

What are the 4 elementary numeric types in ABAP? What is max precision of type P?

Q2 —

Write ABAP that reads `SY-DATUM` and outputs day of week in words.

Q3 —

Define a nested structure: address (street, city, zip) inside a vendor structure. Assign and display.

VENDOR HUB WEEK 1

- Program `ZVH_REPORT_W1` in SE38
- `VENDOR` structure (id, name, city, type)
- Hard-code 3 vendors, `WRITE` each field
- Message class `ZVH_MSG`

DELIVERABLES

- [] M1-M4 notes
- [] Practice programs in SAP system
- [] Vendor Hub Week 1 program running

ABAP Dictionary (DDIC)

Institute-paced • Brochure Modules M5–M8

Master the ABAP Data Dictionary — backbone of all SAP data storage. Learn transparent tables, structures, views, data elements, domains, packages.

BROCHURE MODULES COVERED

M5: Transparent Tables

M6: Structures & Views

M7: Data Elements & Domains

M8: Packages & Object Navigator

Topics to cover by end of segment

- DDIC overview: Transparent vs Pooled vs Cluster tables — when SAP uses each
- Custom Transparent Table: fields, key fields, technical settings
- Structures, Database Views, Projection Views, Maintenance Views
- Data Elements, Domains, value ranges, fixed values
- Packages, Transport Requests, Object Navigator (SE80)
- Search Helps (SE11), Table Maintenance Generator (SE54)

Practice targets

- Browse standard tables MARA, MARC, KNA1 in SE11
- Create ZVHVENDOR (key MANDT+VENDOR_ID; fields NAME, CITY, CATEGORY, STATUS); activate
- Create database view + maintenance view (SM30)
- Create domain ZVHCAT (ELEC/CLOTH/FOOD), data element ZVH_CATEGORY, use in table
- Package ZVHPKG + transport request
- Elementary search help attached to VENDOR_ID

PRACTICE QUESTIONS

Q1 —

What is the difference between a Transparent Table and a Cluster Table? When does SAP use each?

Q2 —

Explain the relationship: Domain → Data Element → Table Field. Why is this separation useful?

Q3 —

Write a WHERE clause query on ZVHVENDOR by CATEGORY + STATUS. What does SY-SUBRC tell you?

VENDOR HUB WEEK 2

- ZVHVENDOR table activated
- ZVH_CATEGORY domain with fixed values
- Search help attached
- Report reads DB via SELECT + WRITE

DELIVERABLES

- [] M5-M8 notes
- [] ZVHVENDOR table in SAP
- [] Domain + Data Element created
- [] SELECT query program working

Search Helps, Lock Objects, Open SQL

Institute-paced • Brochure Modules M9–M12

Master search helps (F4), lock objects (concurrent access), and the full Open SQL syntax including JOINS and subqueries.

BROCHURE MODULES COVERED

M9: Search Helps (F4 Help)

M10: Lock Objects

M11: Open SQL SELECT

M12: Open SQL JOINS & Subqueries

Topics to cover by end of segment

- Elementary vs Collective Search Helps; F4 attachment to fields
- Lock objects: Enqueue/Dequeue FM's, lock modes (S/E/X), SM12
- Open SQL: `SELECT SINGLE` , `SELECT INTO TABLE` , WHERE clauses (EQ, NE, BETWEEN, IN, LIKE)
- JOINS: INNER, LEFT OUTER; `FOR ALL ENTRIES`
- Subqueries: EXISTS, IN, correlated; ORDER BY, GROUP BY, HAVING
- ABAP 7.4+ inline declarations: `DATA(result) = ...` ; performance tips (no `SELECT *`)

Practice targets

- Elementary + collective search helps; attach to `VENDOR_ID`
- Lock object `EZVHENDOR`; `ENQUEUE_EZVHENDOR` before `UPDATE`; watch SM12
- 10 Open SQL queries on `ZVHENDOR`
- 5 JOIN queries + `FOR ALL ENTRIES`
- Vendor Hub milestone: search help on category, lock during update, JOIN with mock order table

PRACTICE QUESTIONS

Q1 —

Difference between elementary and collective search help? Sketch flow for both.

Q2 —

Open SQL query selecting vendors WHERE `STATUS='ACTIVE'` AND `CATEGORY IN ('ELEC','CLOTH')`, ordered by `NAME`, max 50 rows.

Internal Tables, Debugging, Selection Screens

Institute-paced • Brochure Modules M13–M16

Internal tables are the workhorse of ABAP. Master standard/sorted/hashed variants + debugger + selection screens.

MODULES COVERED

M13: Internal Tables (Standard)

M14: Sorted & Hashed Tables

M15: Debugging

M16: Selection Screens

Topics to cover by end of segment

- Standard itab: TYPES , DATA , APPEND , LOOP AT , READ TABLE
- Sorted (WITH UNIQUE KEY), Hashed tables — performance differences
- MODIFY , COLLECT , SORT , DELETE DUPLICATES ; field symbols
- Debugger: breakpoints, watchpoints, call stack, variables, /h
- Selection Screens: PARAMETERS , SELECT-OPTIONS , SELECTION-SCREEN blocks
- Message classes; AT SELECTION-SCREEN event; validations

Practice targets

- Declare itab of ZVHVENDOR; APPEND 5 rows; LOOP + WRITE; READ TABLE; DELETE WHERE
- Convert standard → sorted; try duplicate; convert → hashed; benchmark READ TABLE
- FIELD-SYMBOL in-place modify; SORT BY multi-field; DELETE DUPLICATES BY name
- Debug an earlier program: watchpoints on itab, step through LOOP, use /h
- Vendor Hub milestone: selection screen → itab → WRITE loop, category validation

Q1 —

When to use Hashed vs Sorted vs Standard itab? Give a use case for each.

Q2 —

Read ZVHVENDOR into itab, sort by CITY desc, delete dup by CITY, display unique cities.

Modularization — Subroutines & Function Modules

Institute-paced • Brochure Modules M17–M20

Split monolithic programs into reusable units: subroutines, function modules, includes, packages.

M17: Subroutines (FORM/PERFORM)

M18: Function Modules (SE37)

M19: Function Groups & Includes

M20: Modularization Concepts

Topics to cover by end of segment

- FORM/PERFORM: USING , CHANGING , TABLES parameters; by-value vs by-reference
- Function Modules (SE37): IMPORT / EXPORT / TABLES / CHANGING / EXCEPTIONS
- Function Groups, TOP include, global data
- INCLUDE programs; modular report structure (main, TOP, F01, I01)
- Packages, dependency management, transports (DEV → QAS)
- Exception handling: RAISE , MESSAGE ... RAISING

Practice targets

- Extract Vendor Hub WRITE into FORM; test by-value vs by-reference
- Z_VH_GET_VENDOR : IMPORT VENDOR_ID, EXPORT structure, VENDOR_NOT_FOUND exception
- Add Z_VH_SAVE_VENDOR to same function group; share via TOP include
- Split report into main + TOP + F01 + I01
- Vendor Hub milestone: replace inline logic with FM calls, report = thin orchestrator

Q1 —

Difference between USING and CHANGING in FORM? What does TABLES do?

Q2 —

FM that reads ZVHVENDOR, raises VENDOR_NOT_FOUND. Show EXCEPTIONS block in CALL FUNCTION.

Reports — Classical, Interactive & ALV

Institute-paced • Brochure Modules M21–M24

Everything user-facing in classic ABAP is a report. Master classical output, interactive drill-down, and modern ALV grids (CL_SALV_TABLE).

M21: Classical Reports

M22: Interactive Reports

M23: ALV CL_SALV_TABLE

M24: ALV Grid (REUSE_ALV)

Topics to cover

- Classical output: WRITE formatting, TOP-OF-PAGE , END-OF-PAGE
- Interactive: AT LINE-SELECTION , HIDE , GET CURSOR LINE
- AT USER-COMMAND , function keys, SE41 menu painter
- ALV via CL_SALV_TABLE : factory, columns, hide MANDT, add totals
- ALV events (link_click, double_click), sorting, filtering, handler class
- Classic FM ALV: REUSE_ALV_GRID_DISPLAY

Practice targets

- Format Vendor Hub with headers, alignment, colors
- Drill-down: click vendor → second list shows details
- Rewrite Vendor Hub via CL_SALV_TABLE , add totals
- link_click on VENDOR_ID → popup detail
- Vendor Hub milestone: Main list = CL_SALV_TABLE + drill-down detail

Q1 —

CL_SALV_TABLE for ZVHVENDOR: no MANDT, sortable NAME, total count row.

Q2 —

Explain HIDE in interactive reports. What data + how accessed in AT LINE-SELECTION?

Dialog Programming / Module Pool

Institute-paced • Brochure Modules M25–M28

Build classic SAP GUI transaction screens. Screen Painter, PBO/PAI events, POV/POH F4/F1, tab strips, subscreens.

M25: Screen Painter (SE51)

M26: PBO & PAI

M27: POV & POH

M28: Tab Strips & Subscreens

Topics to cover

- Module pool concept, SE51 screen painter, input fields, table control
- PBO (Process Before Output): title, field properties, populate
- PAI (Process After Input): `AT EXIT-COMMAND`, `FIELD/CHAIN` validation, save via FM
- POV, POH: custom F4 & F1 help
- Table Control: `LOOP AT SCREEN`, scroll handling
- Tab Strips, Subscreens, `CALL SCREEN` / `SET SCREEN` / `LEAVE SCREEN`

Practice targets

- Module pool `ZVH_DIALOG`; Screen 100 with vendor fields + Save/Back
- PBO: title, load vendor data if ID provided
- PAI: validate mandatory, call `Z_VH_SAVE_VENDOR`
- POV for CATEGORY via search help; custom F1 help popup
- Tab strip: Basic Info + Contact tabs as subscreens
- Vendor Hub milestone: full dialog CRUD app (list ALV + detail screen painter)

Q1 —

PBO vs PAI — when does each fire?

Q2 —

`AT EXIT-COMMAND` in PAI: what function code triggers it, why checked before validations?

OO ABAP — Classes, Inheritance, Polymorphism

Institute-paced • Brochure Modules M29–M31

OO ABAP is the modern paradigm. Every new SAP framework (CDS, AMDP, BADIs, BOPF) uses classes. This is where ABAP feels like Java.

M29: Classes & Objects (SE24)

M30: Inheritance & Polymorphism

M31: Interfaces & Events

Topics to cover

- Classes (SE24): PUBLIC/PROTECTED/PRIVATE, methods, constructors
- Static vs instance; Singleton pattern
- Inheritance: `INHERITING FROM`, `REDEFINITION`, super calls
- Polymorphism: parent ref holds child; `CAST + CASE TYPE OF`
- Interfaces: `INTERFACE ... ENDINTERFACE`, implementing
- Events: `EVENTS`, `RAISE EVENT`, `SET HANDLER`

Practice targets

- `ZCL_VENDOR` : attributes, constructor, `GET_DISPLAY_NAME`, `SAVE_TO_DB`
- Static `ZCL_VENDOR⇒GET_INSTANCE` (singleton)
- `ZCL_PREMIUM_VENDOR` inheriting; override `GET_DISPLAY_NAME`
- Interface `ZIVH_SAVEABLE` with `SAVE`; implement in `ZCL_VENDOR` + `ZCL_ORDER`; call polymorphically
- `VENDOR_SAVED` event + handler class logging to a table
- Vendor Hub milestone: OO refactor — `ZCL_VH_REPO` (data) + `ZCL_VH_PRESENTER` (display)

Q1 —

Difference between CLASS-DATA and DATA in ABAP OO? What is CLASS-METHODS?

Q2 —

Implement Observer pattern using `EVENTS` + `SET HANDLER` on `ZCL_VENDOR`.

BDC & LSMW — Data Migration

Institute-paced • Brochure Modules M32–M34

Legacy data migration is a huge chunk of ABAP consulting work. BDC (Batch Data Communication) and LSMW are the tools.

M32: BDC Recording (SHDB)

M33: Call Transaction Method

M34: Session Method & LSMW

Topics to cover

- BDC concept, BDCDATA structure, use cases, failure modes
- SHDB recording (e.g., FK01 vendor create)
- Convert recording to ABAP; `CALL TRANSACTION MODE 'A'/'E'/'N'`
- Session Method: `BDC_OPEN/INSERT/CLOSE_GROUP`, SM35
- Error handling: MESSAGES itab, restart
- LSMW overview: Object, Method, 14 steps, mapping

Practice targets

- SHDB recording of FK01, study BDCDATA
- BDC program: `CALL TRANSACTION 'FK01' USING t_bdcdata`; test all modes
- Session method: 5 vendors in one batch, process via SM35
- LSMW project for ZVH_VENDOR from flat file, map Excel cols
- Vendor Hub milestone: BDC bulk-load 20 vendors via session + error log report

Q1 —

CALL TRANSACTION vs Session Method — when each?

Q2 —

MODE 'E' in CALL TRANSACTION vs MODE 'N' — error handling difference?

Scripts, Smart Forms, RFC, ALE, IDocs, EDI

Institute-paced • Brochure Modules M35–M37

Printouts (Scripts / Smart Forms) + system-to-system integration (RFC / ALE / IDocs / EDI). This segment bridges ABAP to the outside world.

M35: Scripts & Smart Forms

M36: RFC (Remote FC)

M37: ALE / IDocs / EDI

Topics to cover

- SAP Scripts: SE71, `OPEN_FORM` / `WRITE_FORM` / `CLOSE_FORM`
- Smart Forms: SFP, generated FM, data passing, PDF output
- RFC: `DESTINATION` , sRFC / tRFC / aRFC / qRFC, SM59
- ALE: IDoc types, basic/extension, BD54, WE02/WE05
- IDoc inbound: WE19 test tool, mapping, inbound FM
- EDI: X12, EDIFACT, WE20 partner profiles

Practice targets

- Simple invoice layout in SE71; call from ABAP; output to SP01
- Convert SE71 layout → Smart Form; PDF output
- Create RFC-enabled FM; SM59 destination
- Browse ORDERS05 in WE31; monitor WE02
- Vendor Hub milestone: print button → Smart Form PDF vendor summary

Q1 —

sRFC vs tRFC — use case for each.

Q2 —

ALE distribution model: logical system, IDoc flow between 2 systems.

BAPIs, Exits, BADIs, Enhancements, CTS + SAPUI5 Intro

Institute-paced • Brochure Modules M38–M39 + UI5 basics (institute-taught)

Standard SAP hooks: BAPIs (public APIs), User Exits + BADIs + Enhancement Framework (safely modify standard behaviour). CTS (transports). **Plus: institute covers SAPUI5 basics this segment** — that's why we drop the standalone UI5 phase from Part III.

M38: BAPIs

M39: Exits/BADIs/Enhancements/CTS

Bonus: SAPUI5 fundamentals

Topics to cover

- BAPIs: `BAPI_VENDOR_GETDETAIL` / `CREATE` ; RETURN table; BAPI browser
- BAPI + `COMMIT WORK` / `BAPI_TRANSACTION_COMMIT`
- User Exits: SMOD, CMOD; customer + FM exits
- BADIs: SE18 / SE19; definition, implementation, filtering
- Enhancement Framework: implicit vs explicit; source plug-ins
- **SAPUI5 basics (institute-taught):** MVC, XML views, JSONModel, data binding
- CTS / Transport Management: SE10, TR release, import queue

Practice targets

- Call `BAPI_VENDOR_GETDETAIL` ; read RETURN; compare BAPI vs Open SQL
- Call `BAPI_VENDOR_CREATE` + `BAPI_TRANSACTION_COMMIT` ; handle E/W/S messages
- SMOD/CMOD implement one FK01 exit logging vendor create
- Find a vendor BADI; implement; method writes to custom log
- First simple SAPUI5 app in Business Application Studio (follow institute)
- Release all Vendor Hub TRs; simulate DEV → QAS; simple UI5 view over ZVHENDOR

Q1 —

User Exit vs BADI — which is modern SAP approach and why?

Q2 —

Why call `BAPI_TRANSACTION_COMMIT` after creating BAPI? What happens if you don't?

Recommended certifications after this course: (1) SAP Certified Development Associate — ABAP with SAP NetWeaver (**C_TAW12_750**) — the classic ABAP cert. (2) SAP Certified Development Associate — ABAP for SAP HANA 2.0 (**C_HANADEV_18**) after wrap segment. Book these for Jan/Feb 2027 alongside job hunt.

ABAP on HANA — Course Finale

Whenever institute wraps (target: end Nov to mid Dec) • Brochure Module M40

Modern ABAP on HANA: push logic into the DB (code-to-data) via CDS views and AMDP procedures. This is the future of ABAP.

M40: ABAP on HANA — CDS, AMDP, ADBC, Column Store, Code-to-Data

Topics to cover

- HANA column store vs row store; Code-to-Data paradigm; ABAP 7.4+ inline SELECT
- CDS Views: DDL in ADT, `@AbapCatalog` annotations, associations, CASE
- AMDP: class with `IF_AMDP_MARKER_HDB`, SQLScript procedure
- ADBC, SQL Monitor (SQLM)

Practice targets

- Rewrite complex LOOP + aggregation as single inline SELECT; measure LoC reduction
- Create `ZVH_VENDOR_BASIC` CDS view over `ZVH_VENDOR`; add `STATUS_TEXT` via CASE; `@Data.publish`
- `ZCL_VH_AMDP` with SQLScript aggregation; call from ABAP report
- **Vendor Hub Final:** reads from CDS view instead of table; full docs; Part II COMPLETE

Q1 —

What does "code-to-data" mean in ABAP on HANA? How does it differ from traditional ABAP processing?

Q2 —

Create CDS view joining `ZVH_VENDOR` with a custom order table, groups by CITY, outputs count per city. What annotations are needed?

Part II Complete!

SAP Overview (10 days) + ABAP Institute (~11 segments, institute-paced) • Target wrap: end Nov to mid Dec

Vendor Hub: fully functional SAP ABAP application • Next: Part III — SAP BTP + AI (begins Dec 1 self-paced)

SAP BTP + AI — Self-Paced Sprint

Begins Dec 1, 2026 • ~1 month at your pace • 4 phases: BTP Foundation, CAP, AI Core+Joule+RAG, SBPA+Capstone

Part III turns your fresh ABAP skills into full-stack SAP cloud development. You will provision a BTP account, build a CAP backend, layer generative AI on top via SAP AI Core (Claude Sonnet), add HANA Vector RAG, and wire a Joule custom skill + SBPA approval workflow. The capstone **Smart Vendor Hub** combines everything into a demo-ready portfolio project.

Start date is Dec 1 regardless of ABAP status. The ABAP institute may run into mid Dec depending on batch pace. Do NOT wait for ABAP to finish before starting Part III. Instead: begin the BTP self-pace on Dec 1, and let it overlap the tail of ABAP for up to 2 weeks. Reallocate the deep morning slot as needed — institute homework can move to the evening block if a BTP concept is heavy that day.

SAPUI5 note: SAPUI5 is not a separate phase in Part III — you'll have already picked it up during the ABAP institute course (Part II Week 11) and can build simple UI5 screens on top of your CAP service using the Fiori Elements templates in BAS. Focus this month on backend + AI depth.

PHASE	TOPIC	DATES	DAYS	KEY OUTPUT
Ph 13	BTP Foundation	Dec 1-7	7	BTP account, CF, XSUAA, Destination, App Router, BAS
Ph 14	CAP Framework (backend)	Dec 8-15	8	schema.cds, service.cds, custom handlers, deploy to HANA Cloud
Ph 15	SAP AI Core + Joule + RAG	Dec 16-23	8	AI Core endpoint, @sap-ai-sdk, Joule skill, HANA Vector RAG
Ph 16	SBPA + Capstone Smart Vendor Hub	Dec 24-31	8	SBPA workflow, LROP UI, full deployed app, README + demo video

DAILY SCHEDULE

- 8:45 – 11:20 AM — BTP tutorials + coding

- 10:00 – 11:15 PM — Small integrations / documentation

- **Weekends:** 9:00 AM - 2:00 PM study + 6:00 - 9:00 PM capstone work

PREREQUISITES (SHOULD BE DONE)

- Part I capstone (FocusTrack Pro) — Node.js + React

- Part II ABAP course + Vendor Hub in ABAP

- Basic SAPUI5 exposure (institute Week 11)

- BTP free trial or pay-as-you-go account ready

SAP Business Technology Platform

7 days • Dec 1-7, 2026

What is this? BTP is SAP's cloud PaaS — the platform where custom SAP apps, integrations, and AI services live outside the S/4HANA core.

Why should you learn this? Every "modern SAP developer" job requires BTP fluency. Even if you stay ABAP-focused, integrations and Fiori apps land on BTP. This phase sets up the runway for Phases 14-16.

Topics covered: Global account & subaccounts, Cloud Foundry (cf CLI), XSUAA (auth & JWT), Destination service, App Router, Business Application Studio (BAS), manifest.yml, VCAP_SERVICES

DAILY SCHEDULE — STARTING FROM PURE BASICS

DAY	DATE	TOPIC	PRACTICE
Day 1	Tue Dec 1	BTP architecture: global account, subaccounts, directories, service plans	Create BTP free trial. Set up subaccount in Cloud Foundry region. Explore cockpit, entitlements, service marketplace.
Day 2	Wed Dec 2	Cloud Foundry basics: cf CLI, org, space, apps, routes, services	Install cf CLI. <code>cf login</code> . <code>cf push</code> a Node hello-world app. <code>cf env</code> , <code>cf logs</code> .
Day 3	Thu Dec 3	XSUAA: client credentials, user token, scopes, roles	Create XSUAA instance. Bind to app. Decode JWT. Define scopes in xs-security.json.
Day 4	Fri Dec 4	Destination service: HTTP destinations, connectivity, principal propagation	Create destination pointing to your EC2 Node API. Call via <code>@sap/xsenv</code> + <code>@sap/destination-service</code> .
Day 5	Sat Dec 5	App Router: approuter.js, xs-app.json routes, auth flow	Set up <code>@sap/approuter</code> . Route <code>/api</code> to backend, <code>/</code> to UI5 frontend. Deploy to CF. Watch XSUAA redirect.
Day 6	Sun Dec 6	Business Application Studio (BAS): dev space, extensions, templates	Create SAP BAS dev space "Full Stack Cloud Application". Explore Fiori template. Create basic project. Connect to BTP.
Day 7	Mon Dec 7	manifest.yml deep-dive, VCAP_SERVICES, environment binding	Complete manifest.yml for multi-service app. Log and parse VCAP_SERVICES in Node.js to read bound service creds.

YOUTUBE + DOCS

YT SAP Learning "BTP Tutorial": <https://www.youtube.com/@SAPLearning>

DOCS SAP Developers hub tutorials: <https://developers.sap.com> — search "Set Up SAP BTP Trial"

DOCS BTP docs: <https://help.sap.com/docs/btp>

Recommended certification (after Phase 15): SAP Certified Development Associate — SAP BTP Extension Developer. Also free: openSAP "Development on SAP BTP" course.

PRACTICE QUESTIONS

Q1 —

Difference between subaccount and directory in BTP? When would you use each?

Q2 —

Explain XSUAA auth flow: user gets JWT, backend validates. Step-by-step.

Q3 —

What does VCAP_SERVICES contain? Node.js snippet reading XSUAA creds from it.

PHASE 13 DELIVERABLES

- [] BTP trial account set up
- [] CF hello-world deployed
- [] XSUAA instance + JWT decoded
- [] Destination configured
- [] App Router routing working
- [] BAS dev space created

KEY FILES PRODUCED

- manifest.yml (CF deployment)
- xs-security.json (XSUAA scopes)
- xs-app.json (App Router routes)
- .env with local VCAP_SERVICES

SAP Cloud Application Programming Model (CAP)

8 days • Dec 8-15, 2026

What is this? CAP is SAP's opinionated Node.js (or Java) framework for building services on BTP. You declare entities in CDS files; CAP auto-generates OData v4 endpoints, DB deployment, and auth wiring.

Why should you learn this? CAP is the SAP way of building modern services. Understanding CAP = you can write BTP backends, feed Fiori Elements UIs, and hook AI actions — all with less code than pure Node/Express.

Topics covered: cds init, schema.cds (entities/types/associations/compositions), service.cds (expose + restrict), custom handlers (before/on/after), SQLite for local dev, HANA Cloud deploy, mta.yaml multi-target descriptor, cf deploy

DAILY SCHEDULE

DAY	DATE	TOPIC	PRACTICE
Day 1	Tue Dec 8	cds init, schema.cds: entities, types, associations, compositions	Create CAP project: <code>cds init vendor-service</code> . Define Vendors, Categories, Orders. 1:N association. <code>cds watch</code> .
Day 2	Wed Dec 9	service.cds: expose entities, restrict actions, @requires annotation	Create VendorService exposing Vendors + Categories. @requires:'authenticated-user'. Test cds watch + curl.
Day 3	Thu Dec 10	Custom handlers: before/on/after hooks. <code>srv.on('CREATE', ...)</code>	before:CREATE validate name uniqueness. after:READ compute displayStatus. on:approve action.
Day 4	Fri Dec 11	SQLite local dev: <code>cds.requires.db = sqlite</code> . cds deploy --to sqlite	Set up SQLite. Seed data via .csv in db/data/. Full CRUD locally without HANA.
Day 5	Sat Dec 12	mta.yaml: multi-target app descriptor. HANA Cloud + XSUAA modules.	Write mta.yaml: srv (CAP), db (HANA deployer), XSUAA, HDI container. Validate with mbt.
Day 6	Sun Dec 13	cf deploy: build MTA, deploy to CF, HANA Cloud connection	<code>mbt build</code> then <code>cf deploy vendor-service.mtar</code> . Verify cockpit. Test OData with real HANA.
Day 7	Mon Dec 14	Fiori Elements LROP wrapper (institute UI5 skills applied here)	Generate LROP app via BAS template. Add @UI.linItem CDS annotations. Test list + detail auto-generated.
Day 8	Tue Dec 15	End-to-end: LROP → CAP → HANA. Bug bash + docs.	Full stack test. Fix issues. Document endpoints in README.

YOUTUBE + DOCS

DOCS CAP official docs: <https://cap.cloud.sap>

YT SAP Developers "CAP Getting Started" playlist: <https://www.youtube.com/@SAPDevs>

DOCS Fiori Elements docs: <https://experience.sap.com/fiori-design-web/smart-templates/>

Recommended: openSAP "SAP Cloud Application Programming Model" (free, badge on completion).

PRACTICE QUESTIONS

Q1 —

Association vs Composition in CAP CDS? When each?

Q2 —

CAP handler preventing duplicate vendor name (case-insensitive). Show before:CREATE code.

Q3 —

What is mta.yaml? What is HDI container and why does CAP need it on HANA Cloud?

Generative AI on SAP BTP

8 days • Dec 16-23, 2026

What is this? SAP AI Core lets you call LLMs (GPT-4o, Claude Sonnet) through SAP's compliant, enterprise-audited pipe. Joule is SAP's copilot — you can add custom skills to it. HANA Vector Engine powers RAG.

Why should you learn this? This is *the* hottest SAP skill for 2027. Every SAP customer wants "AI-enabled" apps. Being able to say "I've built a CAP action that calls Claude via SAP AI Core with HANA Vector RAG and a Joule skill on top" puts you in the top 5% of SAP applicants.

Topics covered: SAP AI Core setup, model deployment (Claude Sonnet), @sap-ai-sdk/orchestration, prompt templates + content filtering, integrating AI into CAP actions, Joule Studio custom skills, HANA Cloud Vector Engine, embeddings, semantic search, full RAG pipeline

DAILY SCHEDULE

DAY	DATE	TOPIC	PRACTICE
Day 1	Wed Dec 16	SAP AI Core overview, Generative AI Hub, model deployment	Set up AI Core in BTP. Deploy Claude Sonnet endpoint. Test call from Postman. Contrast AI Core vs direct Anthropic API.
Day 2	Thu Dec 17	SAP AI SDK for JavaScript: @sap-ai-sdk/orchestration	Install SDK. Call Claude via orchestration client. Add content filtering. Prompt template with variables.
Day 3	Fri Dec 18	Integrate AI into CAP service: on:action handler calls AI SDK	Add "AI Summary" action to VendorService: takes vendor ID, calls Claude, returns summary.
Day 4	Sat Dec 19	Joule overview, Joule Studio, custom skill development	Open Joule Studio in BAS. Create custom skill calling your CAP AI action. Test in Joule chat.
Day 5	Sun Dec 20	HANA Cloud Vector Engine: embeddings, semantic search, RAG pattern	Create VECTOR table. Store embeddings for vendor descriptions. Semantic similarity search. Feed to AI prompt.
Day 6	Mon Dec 21	Build RAG pipeline: query → embed → vector search → retrieve → prompt → answer	Full RAG pipeline in CAP action: embed question, HANA Vector search, top-3 as context to Claude, return answer.
Day 7	Tue Dec 22	Test + polish AI features. Architecture diagram.	End-to-end test all AI features. Draw: SAPUI5 → CAP → AI SDK → AI Core → Claude. Document prompts.
Day 8	Wed Dec 23	Cost + performance tuning. Prompt caching. Rate limits.	Add prompt caching for repeated prefixes. Log token usage. Add rate limit on CAP AI actions.

YOUTUBE + DOCS

DOCS SAP AI Core docs: <https://help.sap.com/docs/ai-core>

DOCS CAP + AI guide: <https://cap.cloud.sap/docs/guides/ai>

YT SAP Developers "AI Core Tutorial" on YouTube: <https://www.youtube.com/@SAPDevs>

YT "HANA Vector Engine" — SAP HANA Academy on YouTube

Recommended: openSAP "Building Applications with SAP HANA Cloud" + "Generative AI with SAP" (both free, badges).

PRACTICE QUESTIONS

Q1 —

SAP AI Core vs direct Anthropic API — when to choose AI Core?

Q2 —

Describe RAG pattern. How does HANA Cloud Vector Engine enable semantic search?

DELIVERABLES

- [] AI Core endpoint deployed
- [] @sap-ai-sdk/orchestration working
- [] CAP AI Summary action live
- [] Joule custom skill created
- [] Vector table in HANA Cloud
- [] Full RAG pipeline end-to-end

PORTFOLIO IMPACT

- Rare skill combo in 2027 market
- Direct fit for SAP Labs, Accenture, Deloitte AI + SAP roles
- Interview talking points: architecture, cost, safety

Smart Vendor Hub — Final Capstone

8 days • Dec 24-31, 2026 • PART III COMPLETE

What is this? The finale of the whole 6-month journey. Build and deploy *Smart Vendor Hub* — a complete, AI-powered vendor management app combining ABAP RAP backend (Part II), Fiori LROP UI, CAP AI service, and SBPA approval workflow.

Why should you learn this? This project becomes your *primary portfolio artifact* for SAP job applications in Jan/Feb 2027. It demonstrates end-to-end SAP + AI skills that few candidates can match.

Topics covered: SBPA (SAP Build Process Automation) — triggers, steps, decisions, forms, approvers; SBPA API actions; end-to-end integration; deployment to BTP; portfolio README + architecture diagram + demo video

DAILY SCHEDULE

DAY	DATE	TOPIC	PRACTICE
Day 1	Thu Dec 24	SBPA overview: triggers, steps, decisions, forms, approvers	Open SBPA in BTP. Create workflow: trigger on vendor create → manager approval → email → auto-activate on approval.
Day 2	Fri Dec 25	SBPA automation + API actions. Integrate with CAP.	Add API action in SBPA calling CAP to set vendor status ACTIVE after approval. Test full flow.
Day 3	Sat Dec 26	Smart Vendor Hub integration: wire all parts together	Connect: LROP UI (Ph 14) + CAP backend + AI features (Ph 15) + SBPA (Ph 16). Fix integration issues.
Day 4	Sun Dec 27	Full E2E test: create vendor → SBPA approval → AI summary → RAG Q&A → Joule query	Complete user journey. Fix every bug. Add proper error handling.
Day 5	Mon Dec 28	Deploy: MTA to BTP production subaccount, HANA Cloud, App Router	Final deployment. Run deployment checklist. Verify all services healthy.
Day 6	Tue Dec 29	Portfolio docs: README, architecture diagram, demo video script	Comprehensive README. Architecture diagram (draw.io / Lucidchart). Record 3-min demo video.
Day 7	Wed Dec 30	Resume + LinkedIn update. Prep for Jan job search.	Update resume with all 3 portfolio projects (FocusTrack Pro, Vendor Hub ABAP, Smart Vendor Hub). Refresh LinkedIn headline & About.
Day 8	Thu Dec 31	Part III review + Part IV preparation. Roadmap complete!	Review all 16 phases. Push everything to GitHub. Celebrate. Job search starts Jan 2027!

FRONTEND

- SAPUI5 Fiori Elements LROP (vendor list + detail)
- Custom UI5 view (AI chat panel)
- Hosted via App Router on CF
- XSUAA authentication

BACKEND

- CAP VendorService on HANA Cloud
- AI actions: summarize, ask, RAG search
- SBPA approval workflow
- ABAP BAPI via RFC Destination

AI LAYER

- SAP AI Core + Claude Sonnet
- @sap-ai-sdk/orchestration
- HANA Cloud Vector RAG
- Joule custom skill

PROCESS

- SBPA vendor approval workflow
- AI vendor summary generation
- Intelligent RAG search
- Joule conversational queries

End-of-course cert plan for Jan/Feb 2027: (1) SAP Certified Development Associate — SAP BTP Extension Developer. (2) SAP Certified Development Associate — ABAP for SAP HANA 2.0. (3) openSAP badges (free, do all now): CAP, HANA Cloud, Generative AI with SAP.

Study Complete — Dec 31, 2026

6 months • 16 phases • 3 full-stack apps • 1 SAP enterprise application

Next: Part IV — Job Search starts Jan/Feb 2027

Job Hunt Strategy — Jan/Feb 2027

Starting Jan 2027 • Target: SAP ABAP + BTP + AI Developer role in India or KSA

By Jan 2027 you have: 49-day foundation in modern full-stack tech (Jul-Aug 2026), SAP overview + ABAP institute course completed (Sep-Nov 2026), 1-month BTP + AI sprint (Dec 2026), 3 portfolio projects (FocusTrack Pro, ABAP Vendor Hub, Smart Vendor Hub on BTP), and 2+ years of web dev work experience at Mediacle. This section turns that into interviews and offers.

4-WEEK JOB HUNT PLAN

WEEK	DATES (APPROX)	FOCUS	DAILY ACTIVITIES
Week 1	Jan 5-11	Polish & Prepare	Update resume, LinkedIn, GitHub. Record demo videos. Write cover letter templates.
Week 2	Jan 12-18	Apply & Network	Apply 10 jobs/day. Connect with SAP consultants on LinkedIn. Join SAP community groups.
Week 3	Jan 19-25	Interview Prep	Mock interviews daily. Review 90-question bank. Practice whiteboard ABAP coding.
Week 4+	Jan 26+	Convert Offers	Follow up. Negotiate. Attend interview rounds. Keep applying in parallel.

TARGET COMPANIES (TIERED)

TIER 1 DREAM

- SAP Labs India (Bangalore)
- SAP SE (Germany, remote-possible)
- Accenture (SAP Practice)
- Deloitte (SAP CoE)
- IBM Consulting (SAP)

TIER 2 TARGET

- Wipro (SAP Practice)
- TCS (SAP BTP team)
- Infosys (SAP service line)
- HCLTech (SAP)
- Capgemini SAP

TIER 3 BACKUP

- Bristlecone (SAP specialist)
- Yash Technologies
- Birlasoft (SAP)
- Mastech Digital
- In-house SAP teams (banks, MFG)

FOR KSA MARKET (SATA CONNECTION)

- Saudi Aramco (SAP team)
- SABIC (heavy SAP user)
- STC (SAP S/4HANA rollout)
- Accenture KSA / Deloitte KSA
- Local SAP partner firms in Riyadh/Jeddah
- Leverage Telugu/Indian community network in KSA

JOB BOARDS TO MONITOR DAILY

- LinkedIn Jobs (SAP ABAP, SAP BTP, SAP Fiori)
- Naukri.com (India market)
- SAP Community Jobs board
- Indeed India
- Bayt.com (KSA / Middle East)
- SAP PartnerEdge job listings

Resume, Portfolio & LinkedIn Strategy

Jan 5-11, 2027

RESUME STRUCTURE

HEADER

Nikhil Vanama • SAP ABAP + BTP + AI Developer • Bangalore, India • LinkedIn: /in/nikhilvanama • GitHub: /nikhilvanama • Phone: [your number]

SUMMARY (3 LINES MAX)

Full-stack developer transitioning into SAP with 2+ years of Node.js/React experience. Completed SAP ABAP on HANA certification path (Future Labs Technology, 40 modules) and 6-week BTP + AI intensive covering SAPUI5, CAP, SAP AI Core, and Joule. Built "Smart Vendor Hub" — an AI-powered vendor management app deployed on SAP BTP using CAP + Fiori Elements + HANA Vector Engine (RAG).

PORTFOLIO PROJECTS

PROJECT	STACK	WHAT IT DEMONSTRATES
Smart Vendor Hub [PRIMARY SAP PROJECT]	SAPUI5 LROP + CAP + HANA Cloud + SAP AI Core + Claude Sonnet + SBPA + XSUAA	Full-stack SAP BTP development: Fiori Elements, CAP backend, HANA Cloud, AI integration (RAG + Joule), workflow automation (SBPA). Deployed on BTP.
Vendor Hub (ABAP) [ABAP SHOWCASE]	SAP ABAP (SE38/ADT), DDIC (SE11), Module Pool, OO ABAP, ALV, BDC, Smart Forms, CDS Views	End-to-end ABAP application covering reports, dialog programming, OO ABAP, data migration (BDC), output forms, and ABAP on HANA (CDS view).
FocusTrack Pro [FULLSTACK JS]	React 18 + Node.js + Express + MySQL + Claude API + n8n + Docker + AWS EC2	Modern web dev skills: React hooks, REST API with auth (JWT), MySQL with repository pattern, AI agent with tool use, n8n automation, Docker deployment.

LINKEDIN OPTIMIZATION CHECKLIST

- []

Headline: "SAP ABAP + BTP Developer | React + Node.js | AI Integration"
- []

About section: 3-paragraph story (web dev → SAP journey → what you bring)
- []

Featured section: Smart Vendor Hub demo video (3 min)
- []

Experience: Mediacle Technologies updated with SAP/AI skills used
- []

Certifications: Future Labs SAP ABAP completion certificate
- []

Skills: SAP ABAP, SAPUI5, SAP BTP, CAP, HANA, Fiori
- []

Skills: JavaScript, React, Node.js, MySQL, REST API
- []

Skills: Claude API, n8n, Docker, AWS
- []

500+ connections target (connect with SAP professionals)
- []

Post 2x/week: SAP learning journey content (builds visibility)

90-Question Interview Bank

Practice these until you can answer fluently without notes

ABAP QUESTIONS (40)

#	QUESTION	CATEGORY
1	What is the difference between a transparent table, pool table, and cluster table?	DDIC
2	Explain the relationship: Domain → Data Element → Table Field.	DDIC
3	What is the difference between Standard, Sorted, and Hashed internal tables? When do you use each?	Internal Tables
4	What does FOR ALL ENTRIES IN do? What are its dangers?	Open SQL
5	What is the difference between SELECT SINGLE and SELECT UP TO 1 ROWS?	Open SQL
6	Explain PBO and PAI events in module pool programming.	Dialog
7	What is the difference between AT EXIT-COMMAND and AT USER-COMMAND?	Dialog
8	How does the HIDE technique work in interactive reports?	Reports
9	What is CL_SALV_TABLE? How does it differ from REUSE_ALV_GRID_DISPLAY?	ALV
10	What is the difference between a User Exit and a BADI?	Enhancements
11	Explain BDC: what are the two methods (Call Transaction and Session) and when do you use each?	BDC
12	What does BAPI_TRANSACTION_COMMIT do? When must you call it?	BAPIs
13	What is the difference between sRFC, aRFC, tRFC, and qRFC?	RFC
14	Explain the ALE distribution model. What is an IDoc?	ALE/IDoc
15	What is a lock object? How does enqueue/dequeue prevent dirty reads?	Locking
16	What is inheritance in OO ABAP? Show a REDEFINITION example.	OO ABAP
17	What is an ABAP interface? How is it different from a class?	OO ABAP
18	What is a CDS View? How does it differ from a database view?	ABAP on HANA
19	What is "code-to-data" in ABAP on HANA? Why does it improve performance?	ABAP on HANA
20	What is an AMDP? When would you use SQLScript instead of Open SQL?	ABAP on HANA
21	What is a selection screen? What is the difference between PARAMETERS and SELECT-OPTIONS?	Selection Screen
22	What are the events available for selection screen validation (AT SELECTION-SCREEN)?	Selection Screen
23	How do you use FIELD-SYMBOL? What are the advantages over a work area?	Data Objects

#	QUESTION	CATEGORY
24	What is a function group? What is the TOP include?	Modularization
25	What are the EXCEPTIONS in a function module? How do you handle OTHERS?	Function Modules
26	Explain the CTS (Change & Transport System). What is a transport request?	Transport
27	What is SM12? How do you monitor and release locks?	Locking
28	What is a Smart Form? How does it differ from a SAPScript layout set?	Output
29	How do you debug an ABAP program? What is a watchpoint?	Debugging
30	What is SY-SUBRC? When does it return 4?	System Variables
31	What is the difference between APPEND and INSERT in internal tables?	Internal Tables
32	What does DELETE DUPLICATES do? What ORDER BY is required before it?	Internal Tables
33	What is a Search Help? What is the difference between elementary and collective?	DDIC
34	What is a maintenance view? How do you generate table maintenance via SE54?	DDIC
35	What is the difference between ABAP 7.40 inline declarations and classic data declarations?	ABAP Language
36	How do you implement the Singleton pattern in OO ABAP?	OO ABAP
37	What is an Enhancement Spot? Implicit vs explicit enhancement?	Enhancements
38	What is LSMW? What are the 14 steps?	Data Migration
39	How do you read data from multiple tables in one Open SQL SELECT?	Open SQL
40	What is a Collective Search Help? How does a connector field differ from an export field?	DDIC

FIORI / SAPUI5 QUESTIONS (20)

#	QUESTION
1	What is the difference between SAPUI5 MVC pattern and traditional MVC? How does XMLView fit in?
2	Explain the difference between one-way, two-way, and one-time data binding in SAPUI5.
3	What is JSONModel vs ODataModel? When do you use each?
4	How does SAPUI5 routing work? What is a route, target, and view?
5	What is an XMLFragment? How do you load and open a Dialog as a fragment?
6	What is i18n in SAPUI5? How do you access i18n properties in a controller?
7	What is Fiori Elements? What floor plans does it offer?
8	What is a List Report Object Page (LROP)? What annotations drive the UI?
9	What is @UI.lineItem used for? What is @UI.selectionField?

#	QUESTION
10	How do you add a custom action button to a Fiori Elements List Report?
11	What is a Controller Extension in Fiori Elements? Why use it instead of modifying generated code?
12	What is Flexible Column Layout? When do you use 2-column vs 3-column?
13	What is a formatter function in SAPUI5? How do you register and use one?
14	What is the App Router? What does xs-app.json configure?
15	How does SAPUI5 ODataModel V4 differ from V2? What binding types does V4 support?
16	What is @Common.valueList CDS annotation? How does it create F4 help in Fiori Elements?
17	How do you deploy a SAPUI5 app to BTP Cloud Foundry?
18	What is Business Application Studio (BAS)? How does it differ from VS Code?
19	What is element binding in SAPUI5? How do you use it to display a single record?
20	How do you handle errors from ODataModel V4 in a controller (onError, onSuccess)?

BTP / CAP / AI QUESTIONS (20)

#	QUESTION
1	What is SAP BTP? What are its four core areas (Database & Data, Analytics, App Dev, Integration)?
2	What is XSUAA? How does it differ from standard OAuth2 authorization servers?
3	What does VCAP_SERVICES contain? How does a CF app access its bound services?
4	What is the Destination service? Why not hard-code backend URLs?
5	What is CAP? What are the two main files in a CAP project (schema.cds, service.cds)?
6	What is the difference between an association and a composition in CAP CDS?
7	What are custom event handlers in CAP? Explain before/on/after hooks.
8	What is mta.yaml? What is an MTA (Multi-Target Application)?
9	What is an HDI container? Why does CAP on HANA Cloud use it?
10	What is SAP AI Core? How does it relate to Generative AI Hub?
11	What is @sap-ai-sdk/orchestration? How does it differ from calling the Anthropic API directly?
12	What is RAG (Retrieval Augmented Generation)? How does HANA Cloud Vector Engine enable it?
13	What is Joule? What is a Joule custom skill?
14	What is SAP Build Process Automation (SBPA)? What types of automation does it support?
15	What is the difference between Cloud Foundry and Kubernetes runtime on BTP?
16	How do you bind a CF service to your app? What happens to VCAP_SERVICES?
17	What is a CDS View? How can it be exposed as an OData endpoint with @OData.publish?
18	What is HANA Cloud? How is it different from the on-premise HANA database?
19	What is content filtering in SAP AI Orchestration? Why is it important for enterprise AI?
20	Explain the @requires annotation in CAP service.cds. How does it enforce authentication?

SYSTEM DESIGN + BEHAVIORAL QUESTIONS (10)

#	QUESTION	TYPE
1	Design a system to handle 10,000 concurrent users on a SAP BTP app. What services do you use?	System Design
2	How would you migrate 500,000 records from a legacy system to SAP using ABAP? Which method?	Data Migration
3	Tell me about a time you had to learn a completely new technology in a short time. How did you approach it?	Behavioral

#	QUESTION	TYPE
4	Walk me through the Smart Vendor Hub architecture from frontend to database.	Portfolio
5	How would you implement a vendor approval workflow where multiple approvers need to sign off in sequence?	SBPA / Design
6	What is the difference between monolithic SAP deployment and microservices on BTP?	Architecture
7	How do you handle performance issues in an ABAP report that processes 100,000+ records?	ABAP Performance
8	Why are you transitioning from web development to SAP? What unique value do you bring?	Behavioral
9	How would you add AI capabilities to an existing SAP ABAP application without rewriting it?	AI Integration
10	Describe your experience with version control and CI/CD. How does it apply to SAP development?	DevOps / SAP

MOCK INTERVIEW SCHEDULE (WEEK 3)

DAILY SCHEDULE

- 8:00 AM — Answer 5 ABAP questions from memory (no notes)
- 9:00 AM — Answer 3 BTP/CAP questions from memory
- 10:00 AM — Practice whiteboard coding (ABAP internal table manipulation)
- 11:00 AM — Behavioral question practice (STAR method)
- 7:00 PM — Review answers, fill gaps in knowledge

ONLINE MOCK INTERVIEW RESOURCES

- Pramp.com (peer mock interviews)
- SAP Community Q&A practice (answers.sap.com)
- Ask Claude: "Interview me on SAP ABAP basics" (use your own prompt playbook)
- Record yourself answering on phone — watch back for filler words
- Ask a friend/colleague to take the interviewer role

You Got This!

You started as a web developer. You learned prompting, Claude Code, CS fundamentals, full-stack JS, SQL, Docker, AWS, AI agents, SAP ABAP (40 modules), SAPUI5, CAP, SAP AI Core, and shipped 3 projects. That is not a small thing. Trust the process.

Roadmap: Jun 2026 — Dec 2026 • Job Hunt: Jan/Feb 2027